

HelloWorld - Controllers, Actions, Views

- 1. Learning Objectives
- 2. Creating the project
- 3. Controller
- 4. Routes
- 5. Views - Rendering Web Pages
- 6. Views - Adding Dynamic Output

1. Learning Objectives

- creating ASP.NET project
- adding controllers
- working with actions that return a `string`
- working with actions that return a `View`
- communicating between the `Controller` and the `View` using the `ViewBag`
- understanding routes

2. Creating the project

1. Create the ASP.NET project.

To create the project, select **New > Project** from the Visual Studio File menu and select the **Templates > Visual C# > .NET Core** section of the **New Project** dialog window. Select the **ASP.NET Core Web Application (Model-View-Controller)** item, and enter **HelloWorld** into the Name field. Check the “Do not use top-level statements” checkbox.

2. Run the project.

3. Controller

3. This project uses a single controller to select the views displayed to users and to receive form data. Edit the `HomeController.cs` file in the `Controllers` folder to replace the placeholder code provided by Visual Studio with the statements shown below.

```
public class HomeController : Controller
{
    private readonly ILogger<HomeController> _logger;

    public HomeController(ILogger<HomeController> logger)
    {
        _logger = logger;
    }

    public string Index()
    {
```

```

        return "Hello World!";
    }
}

```

4. Run the project and check the result.

4. Routes

MVC applications use the ASP.NET routing system, which decides how URLs map to controllers and actions. A route is a rule that is used to decide how a request is handled.

You can request any of the following URLs, and they will be directed to the Index action on the HomeController:

```

* /
* /Home
* /Home/Index

```

This is a good example of benefiting from following conventions implemented by ASP.NET. In this case, the convention is that we are going to have a controller called HomeController and that this controller will be the starting point for the MVC application. The default configuration that Visual Studio creates for a new project assumes that we will follow this convention. And since we did follow the convention, we automatically got support for the URLs in the preceding list. If we had not followed the convention, we would need to modify the configuration to point to whatever controller we had created instead. For this simple example, the default configuration is all we need.

5. Views - Rendering Web Pages

The output from the current application is just the string Hello World. In order to return an HTML response to a browser request, we need a **View**.

5. Remove all the files in the Views/Home folder.

Views are stored in the Views folder, organized into subfolders. Views that are associated with the Home controller, for example, are stored in a folder called Views/Home. Views that are not specific to a single controller are stored in a folder called Views/Shared.

Visual Studio creates the Home and Shared folders automatically when the Web Application template is used and puts in some placeholder views to get the project started.

6. Modify the Index action in the Home controller as follows.

```

public IActionResult Index()
{
    return View("MyView");
}

```

7. Run the application and notice the error.

Views are stored in the **Views** folder, organized into subfolders. Views that are associated with the **Home** controller, for example, are stored in a folder called **Views/Home**. Views that are not specific to a single controller are stored in a folder called **Views/Shared**.

Visual Studio creates the **Home** and **Shared** folders automatically when the Web Application template is used and puts in some placeholder views to get the project started.

8. Modify the **Index** action in the **Home** controller as follows.

```

public class HomeController : Controller
{
    public ActionResult Index()
    {
        return View();
    }
}

```

9. Add the **Index** view > Razor view files have the **.cshtml** file extension because they are a mix of **C#** code and **HTML** elements.

10. Modify the **Index** view as follows

```

@{
    Layout = null;
}
<!DOCTYPE html>
<html>
<head>
    <meta name="viewport" content="width=device-width" />
    <title>Index</title>
</head>
<body>
    <div>
        Hello World (from the view)
    </div>
</body>
</html>

```

11. Run the application. The view should be displayed.

6. Views - Adding Dynamic Output

One way to pass data from the controller to the view is by using the `ViewBag` object, which is a member of the `Controller` base class. `ViewBag` is a dynamic object to which you can assign arbitrary properties, making those values available in whatever view is subsequently rendered.

12. Modify the `Index` action in the `Home` controller as follows.

```
public IActionResult Index()
{
    int hour = DateTime.Now.Hour;
    ViewBag.Greeting = hour < 12 ? "Good Morning" : "Good Afternoon";
    return View();
}
```

13. Modify the `Index` view as follows.

```
@{
    Layout = null;
}
<!DOCTYPE html>
<html>
<head>
    <meta name="viewport" content="width=device-width" />
    <title>Index</title>
</head>
<body>
    <div>
        @ViewBag.Greeting World (from the view)
    </div>
</body>
</html>
```

HelloWorld - Models, Forms, Model Binding, Validation

- 1. Learning Objectives
- 2. RSVP Application
- 3. Adding a **Model**
- 4. Adding additional actions to the **Controller**
- 5. Linking Action Methods
- 6. Building the Form
- 7. Receiving Form Data
- 8. Using Model Binding
- 9. Storing Responses
- 10. Displaying the Responses
- 11. Adding Validation
- 12. Styling the Content

1. Learning Objectives

- adding a **Model**
- having multiple **Actions** defined in a **Controller**
- communicating between the **Controller** and the **View** using a **Model**
- generating a link towards an **Action** inside a **View**
- configure `<form>` to submit data to controller actions.
- receiving data from forms using **model binding**
- validating form content

2. RSVP Application

Imagine that we want to implement an application that allows people to electronically RSVP (acronym for “Répondez s’il vous plaît”). We would like to have the following features: - a home page that shows information about the event; - a form that can be used to RSVP; - validation for the RSVP form, which will display a thank-you page; - a summary page that shows who is coming to the event.

Note: the application will not persist the answers to any storage.

1. Create a new project named **CourseInvites** using the “ASP.NET Core Web App (Model-View-Controller)” template. Check the “Do not use top-level statements” checkbox.
2. Let’s start with a little cleanup. Remove from the **Views** folder the **Home** and **Shared** folders. Remove from the **Models** folder the **ErrorViewModel** class.
3. Modify the **HomeController** as follows.

```

public class HomeController : Controller
{
    private readonly ILogger<HomeController> _logger;

    public HomeController(ILogger<HomeController> logger)
    {
        _logger = logger;
    }

    public IActionResult Index()
    {
        return View();
    }
}

```

4. Add a view for the `Index` action with the following content.

Razor view files have the `.cshtml` file extension because they are a mix of C# code and HTML elements.

```

@{
    Layout = null;
}
<!DOCTYPE html>
<html>
<head>
    <meta name="viewport" content="width=device-width" />
    <title>ASP.NET lecture!</title>
</head>
<body>
    <div>
        <div>
            We're going to have a lecture on ASP.NET.<br />
        </div>
    </div>
</body>
</html>

```

3. Adding a Model

4. Right-click on the project item in the Solution Explorer window and select `Add > New Folder` from the popup list and set the name of the folder to `Models`.

The **model** is the representation of the real-world objects, processes, and rules that define the subject, known as the domain, of the application. The model, often referred to as a **domain model**, contains the C# objects (known as domain objects)

that make up the universe of the application and the methods that manipulate them. The views and controllers expose the domain to the clients in a consistent manner, and a well-designed MVC application starts with a well-designed model, which is then the focal point as controllers and views are added.

5. Right click on the `Models` folder, select `Add > Class` and create a new class file called `GuestResponse.cs`.

```
public class GuestResponse {  
    public string Name { get; set; }  
    public string Email { get; set; }  
    public string Phone { get; set; }  
    public bool? WillAttend { get; set; }  
}
```

4. Adding additional actions to the Controller

A single controller class can define multiple action methods, and the convention is to group related actions together in the same controller.

1. One of the goals of our app is to include an RSVP form. Let's define an action method that can receive requests for that form. Add the `RsvpForm` action listed below to the `HomeController`.

```
public IActionResult RsvpForm()  
{  
    return View();  
}
```

2. Let's add the corresponding view.

The `@model` Razor expression is used to create a strongly typed view.

```
@model CourseInvites.Models.GuestResponse  
@{  
    Layout = null;  
}  
<!DOCTYPE html>  
<html>  
<head>  
    <meta name="viewport" content="width=device-width" />  
    <title>RsvpForm</title>  
</head>  
<body>  
    <div>  
        This is the RsvpForm.cshtml View  
    </div>
```

```
</body>
</html>
```

5. Linking Action Methods

1. We want to be able to create a link from the `Index` view so that guests can see the `RsvpForm` view without having to know the URL that targets a specific action method. Let's update the `Index` view as follows.

The **asp-action** attribute is an example of a **tag helper attribute**, which is an instruction for Razor that will be performed when the view is rendered. The `asp-action` attribute is an instruction to add a `href` attribute to the `a` element that contains a URL for an action method.

```
@{
    Layout = null;
}
<!DOCTYPE html>
<html>
<head>
    <meta name="viewport" content="width=device-width" />
    <title>ASP.NET lecture!</title>
</head>
<body>
    <div>
        <div>
            We're going to have a lecture on ASP.NET.<br />
        </div>
        <a asp-action="RsvpForm">RSVP Now</a>
    </div>
</body>
</html>
```

You should use the features provided by MVC to generate URLs, rather than hard-code them into your views. When the tag helper created the `href` attribute for the `a` element, it inspected the configuration of the application to figure out what the URL should be. This allows the configuration of the application to be changed to support different URL formats without needing to update any views.

2. Run the project and check the HTML code that has been generated instead of `asp-action="RsvpForm"`.

6. Building the Form

1. Change the contents of the `RsvpForm.cshtml` file as follows

Notice the following tag helper attributes:

- `asp-action` on the form element;
- `asp-for` on the form inputs.

```
@model CourseInvites.Models.GuestResponse
@{
    Layout = null;
}
<!DOCTYPE html>
<html>
<head>
    <meta name="viewport" content="width=device-width" />
    <title>RsvpForm</title>
</head>
<body>
    <form asp-action="RsvpForm" method="post">
        <p>
            <label asp-for="Name">Your name:</label>
            <input asp-for="Name" />
        </p>
        <p>
            <label asp-for="Email">Your email:</label>
            <input asp-for="Email" />
        </p>
        <p>
            <label asp-for="Phone">Your phone:</label>
            <input asp-for="Phone" />
        </p>
        <p>
            <label>Will you attend?</label>
            <select asp-for="WillAttend">
                <option value="">Choose an option</option>
                <option value="true">Yes, I'll be there</option>
                <option value="false">No, I can't come</option>
            </select>
        </p>
        <button type="submit">Submit RSVP</button>
    </form>
</body>
</html>
```

The `@model` expression specifies that the view expects to receive a `GuestResponse` object as its view model.

2. Run the project and check the generated HTML code.

We have defined a `label` and `input` element for each property of

the `GuestResponse` model class. The `asp-for` attribute on the `label` element sets the `value` of the `for` attribute. The `asp-for` attribute on the `input` element sets the `id` and `name` elements.

The `asp-action` tag helper attribute applied to the form element, uses the application's URL routing configuration to set the `action` attribute to a URL that will target a specific action method.

7. Receiving Form Data

1. Update the `HomeController` as follows.

Handling GET and POST requests in separate C# methods helps to keep the controller code tidy, since the two methods have different responsibilities. Both action methods are invoked by the same URL, but MVC makes sure that the appropriate method is called, based on whether we are dealing with a GET or POST request.

```
public class HomeController : Controller {
    private readonly ILogger<HomeController> _logger;
    public HomeController(ILogger<HomeController> logger)
    {
        _logger = logger;
    }

    public IActionResult Index()
    {
        return View();
    }

    [HttpGet]
    public IActionResult RsvpForm() {
        return View();
    }

    [HttpPost]
    public IActionResult RsvpForm(GuestResponse guestResponse) {
        // TODO: store response from guest
        return View();
    }
}
```

8. Using Model Binding

Model binding is a useful MVC feature whereby incoming data is parsed and the key/value pairs in the HTTP request are used to populate properties of domain model types. Model binding is a powerful and customizable feature that eliminates the grind and toil of dealing with HTTP requests directly and lets you work with C# objects rather than dealing with individual data values sent by the browser. The `GuestResponse` object that is passed as the parameter to the action method is automatically populated with the data from the form fields.

9. Storing Responses

1. The project will include a simple in-memory repository to store the responses from users. Add a new class file called `Repository.cs` in the `Models`.

```
public static class Repository
{
    private static List<GuestResponse> responses = new List<GuestResponse>();
    public static IEnumerable<GuestResponse> Responses
    {
        get
        {
            return responses;
        }
    }
    public static void AddResponse(GuestResponse response)
    {
        responses.Add(response);
    }
}
```

2. Let's update the `RsvpForm` method on the `HomeController` class in order to store the response.

```
[HttpPost]
public IActionResult RsvpForm(GuestResponse guestResponse) {
    Repository.AddResponse(guestResponse);
    return View("Thanks", guestResponse);
}
```

3. Let's add the `Thanks.cshtml` view

```
@model CourseInvites.Models.GuestResponse
@{
    Layout = null;
}
<!DOCTYPE html>
<html>
```

```

<head>
  <meta name="viewport" content="width=device-width" />
  <title>Thanks</title>
</head>
<body>
<p>
  <h1>Thank you, @Model.Name!</h1>
  @if (Model.WillAttend == true) {
    @:It's great that you're coming!
  } else {
    @:Sorry to hear that you can't make it, but thanks for letting us know.
  }
</p>
<p>Click <a asp-action="ListResponses">here</a> to see who is coming.</p>
</body>
</html>

```

10. Displaying the Responses

1. Let's add the ListResponses action on the HomeController

```

public IActionResult ListResponses() {
    return View(Repository.Responses.Where(r => r.WillAttend == true));
}

```

2. Add the corresponding view.

```

@model IEnumerable<CourseInvites.Models.GuestResponse>
@{
    Layout = null;
}
<!DOCTYPE html>
<html>
<head>
  <meta name="viewport" content="width=device-width" />
  <title>Responses</title>
</head>
<body>
  <h2>Here is the list of people attending the course</h2>
  <table>
    <thead>
      <tr>
        <th>Name</th>
        <th>Email</th>
        <th>Phone</th>
      </tr>
    </thead>

```

```

        <tbody>
            @foreach (CourseInvites.Models.GuestResponse r in Model) {
                <tr>
                    <td>@r.Name</td>
                    <td>@r.Email</td>
                    <td>@r.Phone</td>
                </tr>
            }
        </tbody>
    </table>
</body>
</html>

```

11. Adding Validation

1. Update the GuestResponse model class as follows.

MVC supports declarative validation rules defined with attributes from the `System.ComponentModel.DataAnnotations` namespace, meaning that validation constraints are expressed using the standard C# attribute features.

Without validation, users could enter nonsense data or even submit an empty form. In an MVC application, you typically apply validation to the domain model rather than in the user interface. This means that you define validation in one place, but it takes effect anywhere in the application that the model class is used.

```

public class GuestResponse {
    [Required(ErrorMessage = "Please enter your name")]
    public string Name { get; set; }
    [Required(ErrorMessage = "Please enter your email address")]
    [RegularExpression(".+\\@.+\\..+",
        ErrorMessage = "Please enter a valid email address")]
    public string Email { get; set; }
    [Required(ErrorMessage = "Please enter your phone number")]
    public string Phone { get; set; }
    [Required(ErrorMessage = "Please specify whether you'll attend")]
    public bool? WillAttend { get; set; }
}

```

Notice that a nullable bool was used for the `WillAttend` property. We did this so that we could apply the `Required` validation attribute. If we had used a regular bool, the value we received through model binding could be only true or false, and we would not be able to tell whether the user had selected a value. A nullable bool has three possible values: true, false, and null. The

browser sends a null value if the user has not selected a value, and this causes the Required attribute to report a validation error. This is a nice example of how MVC elegantly blends C# features with HTML and HTTP.

2. Update the `RsvpForm` action on the `HomeController` as follows. We check to see whether there has been a validation problem using the `ModelState.IsValid` property in the controller class.

`ModelState` is a property of the `Controller` class that provides details of the outcome of the model binding process. If the `ModelState.IsValid` property returns true, then the model binder has not been able to satisfy the validation constraints specified through the attributes on the `GuestResponse` class.

```
[HttpPost]
public IActionResult RsvpForm(GuestResponse guestResponse) {
    if (ModelState.IsValid) {
        Repository.AddResponse(guestResponse);
        return View("Thanks", guestResponse);
    } else {
        // there is a validation error
        return View();
    }
}
```

3. Add a validation summary to the `RsvpForm` view by including the following line.

When it renders a view, Razor has access to the details of any validation errors associated with the request, and tag helpers can access the details to display validation errors to the user.

```
<form asp-action="RsvpForm" method="post">
    <div asp-validation-summary="All"></div>
    ....
```

4. Check how the HTML code for the `<input>` fields changes when the fields contain errors.

The CSS class `.input-validation-error` is added.

5. Change the formatting of the fields that contain errors in the `/css/styles.css` file.

Static content delivered to clients is placed into the `wwwroot` folder and organized by content type so that CSS stylesheets go into the `wwwroot/css` folder, JavaScript files go into the `wwwroot/js` folder.

```
.field-validation-error {color: #f00;}
.field-validation-valid { display: none;}
.input-validation-error { border: 1px solid #f00; background-color: #fee; }
.validation-summary-errors { font-weight: bold; color: #f00;}
.validation-summary-valid { display: none;}
```

6. Reference the stylesheet

Notice that the `wwwroot` folder is omitted from the URL. The default configuration for ASP.NET includes support for serving static content, such as images, CSS stylesheets, and JavaScript files, and it maps requests to the `wwwroot` folder automatically.

```
<link rel="stylesheet" href="~/css/site.css" />
```

12. Styling the Content

Bootstrap is already included in the project. If you are not familiar with it check <http://getbootstrap.com/>

1. Style the `Index.chtml` file as follows

```
@{
    Layout = null;
}
<!DOCTYPE html>
<html>
<head>
    <meta name="viewport" content="width=device-width" />
    <link rel="stylesheet" href="/lib/bootstrap/dist/css/bootstrap.css" />
    <title>Index</title>
</head>
<body>
    <div class="text-center m-2">
        <h3> We're going to have an exciting party!</h3>
        <h4>And YOU are invited!</h4>
        <a class="btn btn-primary" asp-action="RsvpForm">RSVP Now</a>
    </div>
</body>
</html>
```

2. Style the `RsvpForm.chtml` file as follows

```
@model CourseInvites.Models.GuestResponse
@{
    Layout = null;
}
<!DOCTYPE html>
<html>
```

```

<head>
  <meta name="viewport" content="width=device-width" />
  <title>RsvpForm</title>
  <link rel="stylesheet" href="/lib/bootstrap/dist/css/bootstrap.css" />
  <link rel="stylesheet" href="~/css/site.css" />
</head>
<body>
  <form asp-action="RsvpForm" method="post" class="m-2">
    <div asp-validation-summary="All"></div>
    <div class="form-group">
      <label asp-for="Name" class="form-label">Your name:</label>
      <input asp-for="Name" class="form-control" />
    </div>
    <div class="form-group">
      <label asp-for="Email" class="form-label">Your email:</label>
      <input asp-for="Email" class="form-control" />
    </div>
    <div class="form-group">
      <label asp-for="Phone" class="form-label">Your phone:</label>
      <input asp-for="Phone" class="form-control" />
    </div>
    <div class="form-group">
      <label asp-for="WillAttend" class="form-label">
        Will you attend?
      </label>
      <select asp-for="WillAttend" class="form-control">
        <option value="">Choose an option</option>
        <option value="true">Yes, I'll be there</option>
        <option value="false">No, I can't come</option>
      </select>
    </div>
    <button type="submit" class="btn btn-primary mt-3">Submit RSVP</button>
  </form>
</body>
</html>

```

3. Style the Thanks.cstml file as follows

```

<!DOCTYPE html>
<html>
<head>
  <meta name="viewport" content="width=device-width" />
  <title>Thanks</title>
  <link rel="stylesheet" href="/lib/bootstrap/dist/css/bootstrap.css" />
</head>
<body class="text-center">
  <div>

```

```

<h1>Thank you, @Model?.Name!</h1>
@if (Model?.WillAttend == true)
{
    @:It's great that you're coming. The drinks are already in the fridge!
}
else
{
    @:Sorry to hear that you can't make it, but thanks for letting us know.
}
</div>
Click
<a asp-action="ListResponses">here</a> to see who is coming.
</body>
</html>

```

4. Style the ListResponses.cstml file as follows

```

@model IEnumerable<CourseInvites.Models.GuestResponse>
@{
    Layout = null;
}
<!DOCTYPE html>
<html>
<head>
    <meta name="viewport" content="width=device-width" />
    <title>Responses</title>
    <link rel="stylesheet" href="/lib/bootstrap/dist/css/bootstrap.css" />
</head>
<body>
    <div class="text-center p-2">
        <h2 class="text-center">
            Here is the list of people attending the party
        </h2>
        <table class="table table-bordered table-striped table-sm">
            <thead>
                <tr><th>Name</th><th>Email</th><th>Phone</th></tr>
            </thead>
            <tbody>
                @foreach (CourseInvites.Models.GuestResponse r in Model!)
                {
                    <tr>
                        <td>@r.Name</td>
                        <td>@r.Email</td>
                        <td>@r.Phone</td>
                    </tr>
                }
            </tbody>
        </table>
    </div>

```

```
        </table>
    </div>
</body>
</html>
```

Assignment (for you to solve) 1. Use the common “post/redirect/get” pattern for displaying the `Thanks.cshtml` View. Hint: use `TempData` to pass information between actions. `Newtonsoft.Json` might be useful.

MVCStore - Setup and Configuration

- 1. Objectives
- 2. Creating the Projects
- 3. Creating the Folder Structure
- 4. Configuring the Application
- 5. Configuring the Razor View Engine
- 6. Creating the Controller and View
- 7. Bibliography

1. Objectives

- configuring the folder structure;
- configuring services;
- configuring middleware components;
- configuring the Razor View Engine (importing types, layout page);
- creating the default controller, action and view.

2. Creating the Projects

1. To create the project, select **New > Project** from the Visual Studio **File** menu and choose the **ASP.NET Core Empty**. Name the project **MVCStore**. Check the “Do not use top-level statements” checkbox.
2. Run the project. Why do you think that we are seeing the “Hello World!” text?
3. Right-click on the solution item in the Solution Explorer and select **Add > New Project** from the popup menu. Select **xUnit Test Project** from the list of project templates and set the name of the project to **MVCStore.Tests**. Click OK to create the unit test project.
4. Add a reference towards the **MVCStore** project.
5. Install the Moq NuGet package.

3. Creating the Folder Structure

5. Add the following folders

Name	Description
Models	This folder will contain the model classes.
Controllers	This folder will contain the controller classes.
Views	This folder holds everything related to views, including individual Razor files, the view start file, and the view imports file.

4. Configuring the Application

6. Open the `Program` class.
7. Modify the `main` method in the `Program` class as follows in order to enable the MVC framework.

```
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddControllersWithViews();
var app = builder.Build();
//app.MapGet("/", () => "Hello World!");
app.UseStaticFiles();
app.MapDefaultControllerRoute();
app.Run();
```

The `builder.Service` property is used to set up objects, known as services, that can be used throughout the application and that are accessed through dependency injection.

`AddControllersWithViews()` adds the services for controllers. This method configures the MVC services for the commonly used features with controllers with views. It combines the effects of `AddMvcCore(IServiceCollection)`, `AddApiExplorer(IMvcCoreBuilder)`, `AddAuthorization(IMvcCoreBuilder)`, `AddCors(IMvcCoreBuilder)`, `AddDataAnnotations(IMvcCoreBuilder)`, `AddFormatterMappings(IMvcCoreBuilder)`, `AddCacheTagHelper(IMvcCoreBuilder)`, `AddViews(IMvcCoreBuilder)`, and `AddRazorViewEngine(IMvcCoreBuilder)`.

Documenattion: <https://docs.microsoft.com/en-us/dotnet/api/microsoft.extensions.dependencyinject>

The `UseStaticFiles` method enables support for serving static content from the `wwwroot` folder.

ASP.NET Core receives HTTP requests and passes them along a request pipeline, which is populated with middleware components registered using the `app` property. Each middleware component is able to inspect requests, modify them, generate a response, or modify the responses that other components have produced. One especially important middleware component provides the endpoint routing feature, which matches HTTP requests to the application features—known as endpoints—able to produce responses for them. The endpoint routing feature is added to the request pipeline automatically, and the `MapDefaultControllerRoute` registers the MVC Framework as a source of endpoints using a default convention for mapping requests to classes and methods.

5. Configuring the Razor View Engine

The Razor view engine is responsible for processing view files, which have the `.cshtml` extension, to generate HTML responses.

1. Add the Razor View Imports. Right-click the Views folder, select Add > New Item from the pop-up menu, and select the “Razor View Imports” item from the ASP.NET Core > Web > ASP.NET category.

```
@addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers
```

The `@using` statement will allow us to use the types in the `MVCStore.Models` namespace in views without needing to refer to the namespace. The `@addTagHelper` statement enables the built-in tag helpers.

2. Add the Razor View Start.

```
@{  
    Layout = "_Layout";  
}
```

The view start file tells Razor to use a layout file in the HTML that it generates, reducing the amount of duplication in views.

3. Add the Razor Layout. Add a Razor layout file named `_Layout.cshtml` to the Views/Shared folder, with the content show below.

```
<!DOCTYPE html>  
<html>  
<head>  
    <meta name="viewport" content="width=device-width" />  
    <title>@ViewBag.Title</title>  
</head>  
<body>  
    <div>  
        @RenderBody()  
    </div>  
    @await RenderSectionAsync("Scripts", required: false)  
</body>  
</html>
```

This file defines a simple HTML document into which the contents of other views will be inserted by the `@RenderBody` expression.

6. Creating the Controller and View

4. Add a class file named `HomeController.cs` in the `Controllers` folder and use it to define the class shown below. This is a minimal controller that contains just enough functionality to produce a response.

```
public class HomeController: Controller {  
    public IActionResult Index(){  
        return View();  
    }  
}
```

```
}  
}
```

The `MapDefaultControllerRoute` method used in the `Startup` class tells ASP.NET Core how to match URLs to controller classes. The configuration applied by that method declares that the `Index` action method defined by the `Home` controller will be used to handle requests.

The `Index` action method doesn't do anything useful yet and just returns the result of calling the `View` method, which is inherited from the `Controller` base class. This result tells ASP.NET Core to render the default view associated with the action method.

5. Create the view, by adding a Razor View file named `Index.cshtml` to the `Views/Home` folder with the content shown below.

```
<h4>Welcome to MVCStore</h4>
```

6. Run the application.

7. Bibliography

MVCStore - Models, Repository and Database

- 1. Objectives
- 2. Installing the Entity Framework
- 3. Defining the Connection String
- 4. Starting the Data Model
- 5. Creating the Database Context Class
- 6. Configuring Entity Framework Core
- 7. Creating a Repository
- 8. Creating the Database Migration
- 9. Creating Seed Data
- 10. Deleting the database
- 11. Bibliography

1. Objectives

- configuring the types of the columns using attributes;
- creating the Database Context class;
- creating a repository;
- using database migrations;
- seeding data.

2. Installing the Entity Framework

The MVCStore application will store its data in a SQL Server LocalDB database, which is accessed using Entity Framework Core.

Entity Framework Core is the Microsoft object-to-relational mapping (ORM) framework, and it is the most widely used method of accessing databases in ASP.NET Core projects.

1. Install the packages `Microsoft.EntityFrameworkCore.Design` and `Microsoft.EntityFrameworkCore.SqlServer`.

3. Defining the Connection String

2. Create an empty database (example “ism-mvcstore”) using the SQL Server Explorer panel.
3. Modify the `appsettings.json` (or `appsettings.Development.json`) file in order to store the connection string.

A connection string specifies the location and name of the database and provides configuration settings for how the application should connect to the database server.

```
{
  "ConnectionStrings": {
    "DefaultConnection": "...."
```

```
    }
}
```

4. Update the connection string in `appsettings.json`

4. Starting the Data Model

5. Add the following `Product` class to the `Models` folder

```
public class Product
{
    public int ProductID { get; set; }
    public string Name { get; set; }
    public string Description { get; set; }
    [Column(TypeName = "decimal(8, 2)")]
    public decimal Price { get; set; }
    public string Category { get; set; }
}
```

The `Price` property has been decorated with the `Column` attribute to specify the SQL data type that will be used to store values for this property. Not all C# types map neatly onto SQL types, and this attribute ensures the database uses an appropriate type for the application data.

5. Creating the Database Context Class

The database context class is the bridge between the application and the EF Core and provides access to the application's data using model objects.

6. Add a `Data` folder to the project. Add a class file called `ApplicationDbContext.cs` to the `Data` folder and defined the class shown below.

```
public class ApplicationDbContext : DbContext
{
    public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options): base(options)
    {
        public DbSet<Product> Products { get; set; }
    }
}
```

The `DbContext` base class provides access to the Entity Framework Core's underlying functionality, and the `Products` property will provide access to the `Product` table in the database.

6. Configuring Entity Framework Core

Entity Framework Core must be configured so that it knows the type of database to which it will connect, which connection string describes that connection, and which context class will present the data in the database.

7. Modify the `Main` method in the `Program` class as follows.

```
public static void Main(string[] args)
{
    var builder = WebApplication.CreateBuilder(args);
    builder.Services.AddControllersWithViews();
    builder.Services.AddDbContext<ApplicationDbContext>(opts => {
        opts.UseSqlServer(
            builder.Configuration["ConnectionStrings:DefaultConnection"]);
    });
    var app = builder.Build();
    app.UseStaticFiles();
    app.MapDefaultControllerRoute();
    app.Run();
}
```

The `IConfiguration` interface provides access to the ASP.NET Core configuration system, which includes the contents of the `appsettings.json` file. Access to the configuration data is through the `builder.Configuration` property, which allows the database connection string to be obtained. Entity Framework Core is configured with the `AddDbContext` method, which registers the database context class and configures the relationship with the database. The `UseSqlServer` method declares that SQL Server is being used.

Entity Framework Core is configured with the `AddDbContext` method, which registers the database context class and configures the relationship with the database. The `UseSqlServer` method declares that SQL Server is being used and the connection string is read via the `IConfiguration` object.

7. Creating a Repository

The repository pattern is widely used, and it provides a consistent way to access the features presented by the database context class. It can reduce duplication and ensures that operations on the database are performed consistently.

9. In the `Data` folder define the `IStoreRepository` interface as follows.

```
public interface IStoreRepository
{
    IQueryable<Product> Products { get; }
}
```

This interface uses `IQueryable<T>` to allow a caller to obtain a sequence of `Product` objects. The `IQueryable<T>` interface is derived from the more familiar `IEnumerable<T>` interface and

represents a collection of objects that can be queried, such as those managed by a database.

A class that depends on the `IStoreRepository` interface can obtain `Product` objects without needing to know the details of how they are stored or how the implementation class will deliver them.

Understanding IEnumerable and IQueryable Interfaces

The `IQueryable<T>` interface is useful because it allows a collection of objects to be queried efficiently. Later, when we add support for retrieving a subset of `Product` objects from a database, using the `IQueryable<T>` interface allows us to ask the database for just the objects that we require using standard LINQ statements and without needing to know what database server stores the data or how it processes the query. Without the `IQueryable<T>` interface, we would have to retrieve all of the `Product` objects from the database and then discard the ones we don't want, which becomes an expensive operation as the amount of data used by an application increases. It is for this reason that the `IQueryable<T>` interface is typically used instead of `IEnumerable<T>` in database repository interfaces and classes. However, care must be taken with the `IQueryable<T>` interface because each time the collection of objects is enumerated, the query will be evaluated again, which means that a new query will be sent to the database. This can undermine the efficiency gains of using `IQueryable<T>`. In such situations, you can convert `IQueryable<T>` to a more predictable form using the `ToList` or `ToArray` extension method.

10. Create an implementation of the repository interface, named `EFStoreRepository.cs` in the `Models` folder and use it to define the class shown below.

```
public class EFStoreRepository : IStoreRepository {
    private ApplicationDbContext context;

    public EFStoreRepository(ApplicationDbContext ctx) {
        context = ctx;
    }

    public IQueryable<Product> Products {
        get {
            return context.Products;
        }
    }
}
```

The repository implementation maps the `Products` prop-

erty defined by the `IStoreRepository` interface onto the `Products` property defined by the `ApplicationDbContext` class. The `Products` property in the context class returns a `DbSet<Product>` object, which implements the `IQueryable<T>` interface and makes it easy to implement the repository interface when using Entity Framework Core.

11. Add the statement shown below to the `Main` method of the `Program` class to create a service for the `IStoreRepository` interface that uses `EFStoreRepository` as the implementation class
12. ASP.NET Core supports services that allow objects to be accessed throughout the application. One benefit of services is they allow classes to use interfaces without needing to know which implementation class is being used. Application components can access objects that implement the `IStoreRepository` interface without knowing that it is the `EFStoreRepository` implementation class they are using. This makes it easy to change the implementation class the application uses without needing to make changes to the individual components.

```
public static void Main(string[] args)
{
    var builder = WebApplication.CreateBuilder(args);
    builder.Services.AddControllersWithViews();
    builder.Services.AddDbContext<ApplicationDbContext>(opts => {
        opts.UseSqlServer(
            builder.Configuration["ConnectionStrings:DefaultConnection"]);
    });

    // !!!! new/updated code {
    builder.Services.AddScoped<IStoreRepository, EFStoreRepository>();
    //}

    var app = builder.Build();
    app.UseStaticFiles();
    app.MapDefaultControllerRoute();
    app.Run();
}
```

The `AddScoped` method creates a service where each HTTP request gets its own repository object, which is the way that Entity Framework Core is typically used.

8. Creating the Database Migration

Entity Framework Core is able to generate the schema for the database using the data model classes through a feature called mi-

grations. When you prepare a migration, Entity Framework Core creates a C# class that contains the SQL commands required to prepare the database. If you need to modify your model classes, then you can create a new migration that contains the SQL commands required to reflect the changes.

12. Run the following command to generate the initial migration using the **Package Manager Console** panel.

```
---
```

```
Add-Migration Initial
```

```
---
```

> If the command is not recognized, install the package `Microsoft.EntityFrameworkCore.Tools`.

13. Run the following command to update the database.

```
Update-Database
```

14. Check the tables that have been created in the database.

9. Creating Seed Data

14. To populate the database and provide some sample data, let's add a class file called `SeedData.cs` to the `Data` folder.

Futher details: <https://docs.microsoft.com/en-us/aspnet/core/tutorials/razor-pages/sql>

```
public static class SeedData
{
    public static void EnsurePopulated(IApplicationBuilder app)
    {
        ApplicationDbContext context = app.ApplicationServices.CreateScope().ServiceProvider.GetService<ApplicationDbContext>();

        if (context.Database.GetPendingMigrations().Any())
        {
            context.Database.Migrate();
        }

        if (!context.Products.Any())
        {
            context.Products.AddRange(
                new Product
                {
                    Name = "Kayak",
                    Description = "A boat for one person",
                    Category = "Watersports",
                    Price = 275
                }
            );
        }
    }
}
```

```

    },
    new Product
    {
        Name = "Lifejacket",
        Description = "Protective and fashionable",
        Category = "Watersports",
        Price = 48.95m
    },
    new Product
    {
        Name = "Soccer Ball",
        Description = "FIFA-approved size and weight",
        Category = "Soccer",
        Price = 19.50m
    }
);

context.SaveChanges();
}
}
}

```

The static `EnsurePopulated` method receives an `IApplicationBuilder` argument, which is the interface used in the `Main` method of the `Program` class to register middleware components to handle HTTP requests.

`IApplicationBuilder` also provides access to the application's services, including the Entity Framework Core database context service.

The `EnsurePopulated` method obtains a `ApplicationDbContext` object through the `IApplicationBuilder` interface and calls the `Database.Migrate` method if there are any pending migrations, which means that the database will be created and prepared so that it can store `Product` objects. Next, the number of `Product` objects in the database is checked. If there are no objects in the database, then the database is populated using a collection of `Product` objects using the `AddRange` method and then written to the database using the `SaveChanges` method.

15. Call the `EnsurePopulated` in the `Main` method of the `Program` class.

```

public static void Main(string[] args)
{
    var builder = WebApplication.CreateBuilder(args);
    builder.Services.AddControllersWithViews();
    builder.Services.AddDbContext<ApplicationDbContext>(opts => {

```

```

        opts.UseSqlServer(
            builder.Configuration["ConnectionStrings:DefaultConnection"]);
    });

    builder.Services.AddScoped<IStoreRepository, EFStoreRepository>();

    var app = builder.Build();
    app.UseStaticFiles();
    app.MapDefaultControllerRoute();

    // !!!! new/updated code {
    SeedData.EnsurePopulated(app);
    //}

    app.Run();
}

```

10. Deleting the database

Whenever you want to delete the content of the database, you can use the following query.

```

```SQL
/* Azure friendly */
/* Drop all Foreign Key constraints */
DECLARE @name VARCHAR(128)
DECLARE @constraint VARCHAR(254)
DECLARE @SQL VARCHAR(254)

SELECT @name = (SELECT TOP 1 TABLE_NAME FROM INFORMATION_SCHEMA.TABLE_CONSTRAINTS WHERE cons

WHILE @name is not null
BEGIN
 SELECT @constraint = (SELECT TOP 1 CONSTRAINT_NAME FROM INFORMATION_SCHEMA.TABLE_CONSTR
 WHILE @constraint IS NOT NULL
 BEGIN
 SELECT @SQL = 'ALTER TABLE [dbo].[' + RTRIM(@name) + '] DROP CONSTRAINT [' + RTRIM(@
 EXEC (@SQL)
 PRINT 'Dropped FK Constraint: ' + @constraint + ' on ' + @name
 SELECT @constraint = (SELECT TOP 1 CONSTRAINT_NAME FROM INFORMATION_SCHEMA.TABLE_CON
 END
 SELECT @name = (SELECT TOP 1 TABLE_NAME FROM INFORMATION_SCHEMA.TABLE_CONSTRAINTS WHERE cons
END
GO

/* Drop all Primary Key constraints */

```

```

DECLARE @name VARCHAR(128)
DECLARE @constraint VARCHAR(254)
DECLARE @SQL VARCHAR(254)

SELECT @name = (SELECT TOP 1 TABLE_NAME FROM INFORMATION_SCHEMA.TABLE_CONSTRAINTS WHERE cons

WHILE @name IS NOT NULL
BEGIN
 SELECT @constraint = (SELECT TOP 1 CONSTRAINT_NAME FROM INFORMATION_SCHEMA.TABLE_CONSTR
 WHILE @constraint is not null
 BEGIN
 SELECT @SQL = 'ALTER TABLE [dbo].[' + RTRIM(@name) + '] DROP CONSTRAINT [' + RTRIM(@
 EXEC (@SQL)
 PRINT 'Dropped PK Constraint: ' + @constraint + ' on ' + @name
 SELECT @constraint = (SELECT TOP 1 CONSTRAINT_NAME FROM INFORMATION_SCHEMA.TABLE_CON
 END
 SELECT @name = (SELECT TOP 1 TABLE_NAME FROM INFORMATION_SCHEMA.TABLE_CONSTRAINTS WHERE cons
END
GO

/* Drop all tables */
DECLARE @name VARCHAR(128)
DECLARE @SQL VARCHAR(254)

SELECT @name = (SELECT TOP 1 [name] FROM sysobjects WHERE [type] = 'U' AND category = 0 ORDE

WHILE @name IS NOT NULL
BEGIN
 SELECT @SQL = 'DROP TABLE [dbo].[' + RTRIM(@name) + ']'
 EXEC (@SQL)
 PRINT 'Dropped Table: ' + @name
 SELECT @name = (SELECT TOP 1 [name] FROM sysobjects WHERE [type] = 'U' AND category = 0
END
GO

```

## 11. Bibliography

## Controllers, Actions and Views

- 1. Objectives
- 2. Preparing the Controller
- 3. Unit Testing the HomeController
- 4. Updating the View
- 5. Displaying Page Links
- 6. Bibliography

### 1. Objectives

- unit testing the actions in the controllers;
- declaring tag helpers;
- adding a new route;

### 2. Preparing the Controller

1. Modify the HomeController as highlighted below.

```
public class HomeController : Controller
{
 private IStoreRepository repository;
 public HomeController(IStoreRepository repo)
 {
 repository = repo;
 }
 public IActionResult Index()
 {
 return View(repository.Products);
 }
}
```

When ASP.NET Core needs to create a new instance of the `HomeController` class to handle an HTTP request, it will inspect the constructor and see that it requires an object that implements the `IStoreRepository` interface. To determine what implementation class should be used, ASP.NET Core consults the configuration in the `Startup` class, which tells it that `EFStoreRepository` should be used and that a new instance should be created for every request. ASP.NET Core creates a new `EFStoreRepository` object and uses it to invoke the `HomeController` constructor to create the controller object that will process the HTTP request. This is known as **dependency injection**, and its approach allows the `HomeController` object to access the application's repository through the `IStoreRepository` interface without knowing which implementation class has been configured. I could recon-

figure the service to use a different implementation class—one that doesn't use Entity Framework Core, for example—and dependency injection means that the controller will continue to work without changes.

### 3. Unit Testing the HomeController

2. We can unit test that the controller is accessing the repository correctly by creating a mock repository, injecting it into the constructor of the `HomeController` class, and then calling the `Index` method to get the response that contains the list of products. We then compare the `Product` objects we get to what we would expect from the test data in the mock implementation.

```
public class HomeControllerTests
{
 [Fact]
 public void Can_Use_Repository()
 {
 // Arrange
 Mock<IStoreRepository> mock = new Mock<IStoreRepository>();
 mock.Setup(m => m.Products).Returns(
 (new Product[] {
 new Product {ProductID = 1, Name = "P1"},
 new Product {ProductID = 2, Name = "P2"}
 }).AsQueryable<Product>()
);
 HomeController controller = new HomeController(mock.Object);

 // Act
 IEnumerable<Product>? result = (controller.Index() as ViewResult)?.ViewData.Model;

 // Assert
 Product[] prodArray = result?.ToArray() ?? Array.Empty<Product>();
 Assert.True(prodArray.Length == 2);
 Assert.Equal("P1", prodArray[0].Name);
 Assert.Equal("P2", prodArray[1].Name);
 }
}
```

### 4. Updating the View

3. The `Index` action method passes the collection of `Product` objects from the repository to the `View` method, which means these objects will be the view model that Razor uses when it generates HTML content from the view. Make the changes to the view shown below to generate content using the

Product view model objects.

```
@model IQueryable<Product>

@foreach (var p in Model ?? Enumerable.Empty<Product>()) {
 <div>
 <h3>@p.Name</h3>
 @p.Description
 <h4>@p.Price.ToString("c")</h4>
 </div>
}
```

Note: You might need to add the following line to the `_ViewImports.cshtml` file.

```
@using MVCStore.Models
```

The `@model` expression at the top of the file specifies that the view expects to receive a sequence of `Product` objects from the action method as its model data. We use an `@foreach` expression to work through the sequence and generate a simple set of HTML elements for each `Product` object that is received. The view doesn't know where the `Product` objects came from, how they were obtained, or whether they represent all the products known to the application. Instead, the view deals only with how details of each `Product` are displayed using HTML elements.

4. Add support for pagination so that the view displays a smaller number of products on a page, and the user can move from page to page to view the overall catalog. Update the `Index` method on the `HomeController` as follows.

```
public int PageSize = 2;
public IActionResult Index(int productPage = 1)
{
 var products = repository.Products
 .OrderBy(p => p.ProductID)
 .Skip((productPage - 1) * PageSize)
 .Take(PageSize);

 return View(products);
}
```

The `PageSize` field specifies the number of products per page. We have added an optional parameter to the `Index` method, which means that if the method is called without a parameter, the call is treated as though we had supplied the value specified in the parameter definition, with the effect that the action method

displays the first page of products when it is invoked without an argument. Within the body of the action method, we get the `Product` objects, order them by the primary key, skip over the products that occur before the start of the current page, and take the number of products specified by the `PageSize` field.

5. Unit test the pagination feature in the `HomeControllerTests` class.

```
[Fact]
public void Can_Paginate()
{
 // Arrange
 Mock<IStoreRepository> mock = new Mock<IStoreRepository>();

 mock.Setup(m => m.Products).Returns((new Product[] {
 new Product {ProductID = 1, Name = "P1"},
 new Product {ProductID = 2, Name = "P2"},
 new Product {ProductID = 3, Name = "P3"},
 new Product {ProductID = 4, Name = "P4"},
 new Product {ProductID = 5, Name = "P5"}
 }).AsQueryable<Product>());

 HomeController controller = new HomeController(mock.Object);
 controller.PageSize = 3;

 // Act
 IEnumerable<Product> result = (controller.Index(2) as ViewResult)?.ViewData.Model as IEnumerable<Product>;

 // Assert
 Product[] prodArray = result.ToArray();
 Assert.True(prodArray.Length == 2);
 Assert.Equal("P4", prodArray[0].Name);
 Assert.Equal("P5", prodArray[1].Name);
}
```

## 5. Displaying Page Links

To support the tag helper that will display the page numbers, we need to pass information to the view about the number of pages available, the current page, and the total number of products in the repository. The easiest way to do this is to create a view model class, which is used specifically to pass data between a controller and a view.

6. Create a `ViewModels` folder in your project. Add a class called `PagingInfoViewModel`.

```
public class PagingInfoViewModel
{
```

```

 public int TotalItems { get; set; }
 public int ItemsPerPage { get; set; }
 public int CurrentPage { get; set; }
 public int TotalPages
 {
 get
 {
 return (int)Math.Ceiling((decimal)TotalItems / ItemsPerPage);
 }
 }
}

```

7. Let's now create the tag helper that will display the page numbers. Create a folder named `Infrastructure` and add to it a class called `PageLinkTagHelper`

```

[HtmlTargetElement("div", Attributes = "page-model")]
public class PageLinkTagHelper : TagHelper
{
 private IUrlHelperFactory urlHelperFactory;
 public PageLinkTagHelper(IUrlHelperFactory helperFactory)
 {
 urlHelperFactory = helperFactory;
 }
 [ViewContext]
 [HtmlAttributeNotBound]
 public ViewContext? ViewContext { get; set; }
 public PagingInfoViewModel? PageModel { get; set; }
 public string? PageAction { get; set; }
 public override void Process(TagHelperContext context,
 TagHelperOutput output)
 {
 if (ViewContext != null && PageModel != null)
 {
 IUrlHelper urlHelper = urlHelperFactory.GetUrlHelper(ViewContext);
 TagBuilder result = new TagBuilder("div");
 for (int i = 1; i <= PageModel.TotalPages; i++)
 {
 TagBuilder tag = new TagBuilder("a");
 tag.Attributes["href"] = urlHelper.Action(PageAction,
 new { productPage = i });

 tag.InnerHtml.Append(i.ToString());
 result.InnerHtml.AppendHtml(tag);
 }
 output.Content.AppendHtml(result.InnerHtml);
 }
 }
}

```

```

 }
}

```

This tag helper populates a `div` element with a elements that correspond to pages of products. Tag helpers are one of the most useful ways that you can introduce C# logic into your views. The code for a tag helper can look tortured because C# and HTML don't mix easily. But using tag helpers is preferable to including blocks of C# code in a view because a tag helper can be easily unit tested.

8. Register the new tag helper in the `_ViewImports.cshtml` file.

```

@addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers
// !!!! new/updated code {
@addTagHelper *, MVCStore
//}
@using MVCStore.Models
// !!!! new/updated code {
@using MVCStore.ViewModels
//}

```

9. Add a new class in the `MVCStore.Tests` project for unit testing the `PageLinkTagHelper` tag helper class.

```

public class PageLinkTagHelperTests
{
 [Fact]
 public void Can_Generate_Page_Links()
 {
 // Arrange
 var urlHelper = new Mock<IUrlHelper>();
 urlHelper.SetupSequence(x => x.Action(It.IsAny<UrlActionContext>()))
 .Returns("Test/Page1")
 .Returns("Test/Page2")
 .Returns("Test/Page3");

 var urlHelperFactory = new Mock<IUrlHelperFactory>();
 urlHelperFactory.Setup(f =>
 f.GetUrlHelper(It.IsAny<ActionContext>()))
 .Returns(urlHelper.Object);
 PageLinkTagHelper helper =
 new PageLinkTagHelper(urlHelperFactory.Object)
 {
 PageModel = new PagingInfo
 {
 CurrentPage = 2,
 TotalItems = 28,
 }
 }
 }
}

```

```

 ItemsPerPage = 10
 },
 PageAction = "Test"
};
TagHelperContext ctx = new TagHelperContext(
 new TagHelperAttributeList(),
 new Dictionary<object, object>(), "");
var content = new Mock<TagHelperContent>();
TagHelperOutput output = new TagHelperOutput("div",
 new TagHelperAttributeList(),
 (cache, encoder) => Task.FromResult(content.Object));
// Act
helper.Process(ctx, output);
// Assert
Assert.Equal(@"1"
+ @"2"
+ @"3",
 output.Content.GetContent());
}
}

```

The complexity in this test is in creating the objects that are required to create and use a tag helper. Tag helpers use `IUrlHelperFactory` objects to generate URLs that target different parts of the application, and I have used Moq to create an implementation of this interface and the related `IUrlHelper` interface that provides test data.

10. We need to provide an instance of the `PagingInfo` view model class to the view. Add a new class called `ProductsListViewModel` to the `ViewModels` folder.

```

public class ProductsListViewModel
{
 public IEnumerable<Product> Products { get; set; } = Enumerable.Empty<Product>();
 public PagingInfoViewModel PagingInfo { get; set; } = new();
}

```

11. Update the `Index` action in the `HomeController` class to use the `ProductsListViewModel` class in order to provide the view with details of the products to display on the page and with details of the pagination, as shown below.

```

public IActionResult Index(int productPage = 1)
{
 var viewModel = new ProductsListViewModel
 {
 Products = repository.Products
 }
}

```

```

 .OrderBy(p => p.ProductID)
 .Skip((productPage - 1) * PageSize)
 .Take(PageSize),
 PagingInfo = new PagingInfo
 {
 CurrentPage = productPage,
 ItemsPerPage = PageSize,
 TotalItems = repository.Products.Count()
 }
 };

 return View(viewModel);
}

```

These changes pass a `ProductsListViewModel` object as the model data to the view.

12. Modify the earlier unit tests to reflect the new result from the Index action method.

```

[Fact]
public void Can_Use_Repository()
{
 // Arrange
 Mock<IStoreRepository> mock = new Mock<IStoreRepository>();
 mock.Setup(m => m.Products).Returns(
 (new Product[] {
 new Product {ProductID = 1, Name = "P1"},
 new Product {ProductID = 2, Name = "P2"}
 }).AsQueryable<Product>()
);

 HomeController controller = new HomeController(mock.Object);

 // Act
 // !!!! new/updated code {
 ProductsListViewModel result = controller.Index()?.ViewData.Model as ProductsListVi
 // }

 // Assert
 // !!!! new/updated code {
 Product[] prodArray = result.Products.ToArray();
 // }
 Assert.True(prodArray.Length == 2);
 Assert.Equal("P1", prodArray[0].Name);
 Assert.Equal("P2", prodArray[1].Name);
}

[Fact]

```

```

public void Can_Paginate()
{
 // Arrange
 Mock<IStoreRepository> mock = new Mock<IStoreRepository>();

 mock.Setup(m => m.Products).Returns((new Product[] {
 new Product {ProductID = 1, Name = "P1"},
 new Product {ProductID = 2, Name = "P2"},
 new Product {ProductID = 3, Name = "P3"},
 new Product {ProductID = 4, Name = "P4"},
 new Product {ProductID = 5, Name = "P5"}
 }).AsQueryable<Product>());

 HomeController controller = new HomeController(mock.Object);
 controller.PageSize = 3;

 // Act
 // !!!! new/updated code {
 ProductsListViewModel result = controller.Index(2)?.ViewData.Model as ProductsList
 // }

 // Assert
 // !!!! new/updated code {
 Product[] prodArray = result.Products.ToArray();
 // }
 Assert.True(prodArray.Length == 2);
 Assert.Equal("P4", prodArray[0].Name);
 Assert.Equal("P5", prodArray[1].Name);
}

```

13. Add a unit test in order to check if controller sends the correct pagination data to the view in the class HomeControllerTests.

```

[Fact]
public void Can_Send_Pagination_View_Model()
{
 // Arrange
 Mock<IStoreRepository> mock = new Mock<IStoreRepository>();
 mock.Setup(m => m.Products).Returns((new Product[] {
 new Product {ProductID = 1, Name = "P1"},
 new Product {ProductID = 2, Name = "P2"},
 new Product {ProductID = 3, Name = "P3"},
 new Product {ProductID = 4, Name = "P4"},
 new Product {ProductID = 5, Name = "P5"}
 }).AsQueryable<Product>());

 // Arrange
 HomeController controller =

```

```

new HomeController(mock.Object) { PageSize = 3 };
// Act
ProductsListViewModel result = controller.Index(2)?.ViewData.Model as ProductsListV
// Assert
PagingInfo pageInfo = result.PagingInfo;
Assert.Equal(2, pageInfo.CurrentPage);
Assert.Equal(3, pageInfo.ItemsPerPage);
Assert.Equal(5, pageInfo.TotalItems);
Assert.Equal(2, pageInfo.TotalPages);
}

```

14. Update the Index.cshtml corresponding to the HomeController to use the new viewmodel type.

```

@model ProductsListViewModel
@foreach (var p in Model?.Products ?? Enumerable.Empty<Product>())
{
 <div>
 <h3>@p.Name</h3>
 @p.Description
 <h4>@p.Price.ToString("c")</h4>
 </div>
}

```

We have changed the @model directive to tell Razor that we are now working with a different data type.

15. Display the links towards the pages in the Index.cshtml corresponding to the HomeController.

```

@model ProductsListViewModel

@foreach (var p in Model.Products)
{
 <div>
 <h3>@p.Name</h3>
 @p.Description
 <h4>@p.Price.ToString("c")</h4>
 </div>
}

<div page-model="@Model?.PagingInfo" page-action="Index"></div>

```

When Razor finds the page-model attribute on the div element, it asks the PageLinkTagHelper class to transform the element.

16. Let's improve the urls using the ASP.NET Core routing feature. Modify the Main method of the Program class as follows.

```

public static void Main(string[] args)

```

```

{
 var builder = WebApplication.CreateBuilder(args);
 builder.Services.AddControllersWithViews();
 builder.Services.AddDbContext<ApplicationDbContext>(opts => {
 opts.UseSqlServer(
 builder.Configuration["ConnectionStrings:DefaultConnection"]);
 });

 builder.Services.AddScoped<IStoreRepository, EFStoreRepository>();

 var app = builder.Build();
 app.UseStaticFiles();
 // !!!! new/updated code {
 app.MapControllerRoute("pagination",
 "Products/Page{productPage}",
 new { Controller = "Home", action = "Index" });
 //}
 app.MapDefaultControllerRoute();

 SeedData.EnsurePopulated(app);

 app.Run();
}

```

It is important that you add the new route before the call to the `MapDefaultControllerRoute` method. The routing system processes routes in the order they are listed, and we need the new route to take precedence over the existing one.

This is the only alteration required to change the URL scheme for product pagination. ASP.NET Core and the routing function are tightly integrated, so the application automatically reflects a change like this in the URLs used by the application, including those generated by tag helpers.

## 6. Bibliography

## Styling, PartialViews

- 1. Objectives
- 2. Applying Bootstrap Styles
- 3. Partial Views
- 4. Bibliography

### 1. Objectives

- adding client-side libraries;
- styling with Bootstrap;
- creating and using partial views.

### 2. Applying Bootstrap Styles

1. Add a new folder called `wwwroot` to the project.
2. Add the “Bootstrap” framework to your project by right clicking on the project and choosing “Add” > “Client Side Library”. Use “cdnjs” as a provider and search for “twitter” in order to install version 5.x.
3. Razor layouts provide common content so that it doesn’t have to be repeated in multiple views. Update the `_Layout.cshtml` file in the `Views/Shared` folder to include the Bootstrap CSS stylesheet in the content sent to the browser and define a common header that will be used throughout the application

```
<!DOCTYPE html>

<html>
<head>
 <meta name="viewport" content="width=device-width" />
 <title>@ViewBag.Title</title>

 <!-- !!!! new/updated code { -->
 <link href="/lib/bootstrap/css/bootstrap.min.css" rel="stylesheet" />
 <!-- } -->
</head>
<body>
 <!-- !!!! new/updated code { -->
 <div class="bg-dark text-white p-2">
 MVC STORE
 </div>
 <div class="row m-1 p-1">
 <div id="categories" class="col-3">
 Put something useful here later
 </div>
 <div class="col-9">
```

```

 @RenderBody()
 </div>
</div>
<!-- } -->
 @await RenderSectionAsync("Scripts", required: false)
</body>
</html>

```

4. Also update the styling applied to the `Index.cshtml` file.

```

@model ProductsListViewModel

@foreach (var p in Model.Products)
{
 <div class="card card-outline-primary m-1 p-1">
 <div class="bg-faded p-1">
 <h4>
 @p.Name

 <small>@p.Price.ToString("c")</small>

 </h4>
 </div>
 <div class="card-text p-1">@p.Description</div>
 </div>
}

<div page-model="@Model.PagingInfo"
 page-action="Index"
 page-classes-enabled="true"
 page-class="btn"
 page-class-normal="btn-outline-dark"
 page-class-selected="btn-primary"
 class="btn-group pull-right m-1">
</div>

```

5. We need to style the buttons generated by the `PageLinkTagHelper` class, but we don't want to hardwire the Bootstrap classes into the C# code because it makes it harder to reuse the tag helper elsewhere in the application or change the appearance of the buttons. Instead, we have defined custom attributes on the div element that specify the classes that we require, and these correspond to properties we added to the tag helper class, which are then used to style the a elements that are produced.

```

[HtmlTargetElement("div", Attributes = "page-model")]
public class PageLinkTagHelper : TagHelper
{
 private IUrlHelperFactory urlHelperFactory;

```

```

public PageLinkTagHelper(IUrlHelperFactory helperFactory)
{
 urlHelperFactory = helperFactory;
}
[ViewContext]
[HtmlAttributeNotBound]
public ViewContext? ViewContext { get; set; }
public PagingInfo? PageModel { get; set; }
public string? PageAction { get; set; }

// new/updated code {
public bool PageClassesEnabled { get; set; } = false;
public string PageClass { get; set; } = String.Empty;
public string PageClassNormal { get; set; } = String.Empty;
public string PageClassSelected { get; set; } = String.Empty;
//}

public override void Process(TagHelperContext context,
TagHelperOutput output)
{
 if (ViewContext != null && PageModel != null)
 {
 IUrlHelper urlHelper = urlHelperFactory.GetUrlHelper(ViewContext);
 TagBuilder result = new TagBuilder("div");
 for (int i = 1; i <= PageModel.TotalPages; i++)
 {
 TagBuilder tag = new TagBuilder("a");
 tag.Attributes["href"] = urlHelper.Action(PageAction, new { productPage

 // new/updated code {
 if (PageClassesEnabled)
 {
 tag.AddCssClass(PageClass);
 tag.AddCssClass(i == PageModel.CurrentPage
 ? PageClassSelected : PageClassNormal);
 }
 //}

 tag.InnerHtml.Append(i.ToString());
 result.InnerHtml.AppendHtml(tag);
 }
 output.Content.AppendHtml(result.InnerHtml);
 }
}
}

```

The values of the attributes are automatically used to set the tag helper property values, with the mapping between the HTML attribute name format (page-class-normal) and the C# property name format (PageClassNormal) taken into account. This allows tag helpers to respond differently based on the attributes of an HTML element, creating a more flexible way to generate content in an ASP.NET Core application.

### 3. Partial Views

6. Creating a Partial View. We would like to refactor the application to simplify the `Index.cshtml` view. We are going to create a partial view, which is a fragment of content that we can embed into another view, rather like a template. They help reduce duplication when you need the same content to appear in different places in an application. To create the partial view, we added a Razor View called `ProductSummary.cshtml` to the `Views/Shared` folder and added the markup shown below.

```
@model Product
<div class="card card-outline-primary m-1 p-1">
 <div class="bg-faded p-1">
 <h4>
 @Model?.Name
 <span class="badge rounded-pill bg-primary text-white"
 style="float:right">
 <small>@Model?.Price.ToString("c")</small>

 </h4>
 </div>
 <div class="card-text p-1">@Model?.Description</div>
</div>
```

7. Update the `Index.cshtml` file in the `Views/Home` folder so that it uses the partial view.

```
@model ProductsListViewModel

@foreach (var p in Model?.Products ?? Enumerable.Empty<Product>())
{
 <partial name="ProductSummary" model="p" />
}

<div page-model="@Model?.PagingInfo" page-action="Index" page-classes-enabled="true"
 page-class="btn" page-class-normal="btn-outline-dark"
 page-class-selected="btn-primary" class="btn-group pull-right m-1">
</div>
```

We have taken the markup that was previously in the `@foreach`

expression in the `Index.cshtml` view and moved it to the new partial view. We call the partial view using a partial element, using the `name` and `model` attributes to specify the name of the partial view and its view model. Using a partial view allows the same markup to be inserted into any view that needs to display a summary of a product.

## 4. Bibliography

# CRUD - Create, Read, Update and Delete

- 1. Objectives
- 2. Creating a CRUD Controller
- 3. Displaying the Products
- 4. Editing the Products
- 5. Creating New Products
- 6. Deleting Products
- 7. Bibliography

## 1. Objectives

- retrieving items from the database;
- adding new items to the database;
- updating existing items in the database;
- deleting existing item from the database;

## 2. Creating a CRUD Controller

1. Add a new controller to the **Controllers** folder called **AdminController**

```
public class AdminController : Controller
{
 private IStoreRepository repository;
 public AdminController(IStoreRepository repo)
 {
 repository = repo;
 }
 public IActionResult Index()
 {
 return View(repository.Products);
 }
}
```

The controller constructor declares a dependency on the **IStoreRepository** interface, which will be resolved when instances are created. The controller defines a single action method, **Index**, that calls the **View** method to select the default view for the action, passing the set of products in the database as the view model.

## 3. Displaying the Products

2. Add a new layout file, called **\_AdminLayout**, to the **Views/Shared** folder with the following code.

```
<!DOCTYPE html>
```

```

<html lang="en">
<head>
 <meta name="viewport" content="width=device-width, initial-scale=1, shrink-to-fit=no">

 <title>@ViewBag.Title</title>

 <link href="/lib/bootstrap/dist/css/bootstrap.min.css" rel="stylesheet" />
</head>
<body>
 <nav class="navbar bg-light navbar-expand-sm">
 <div class="container-fluid">
 MVCStore
 </div>
 </nav>

 <div class="container-fluid">
 @RenderBody()
 </div>

 @await RenderSectionAsync("Scripts", required: false)
</body>
</html>

```

For navbars that never collapse, add the `.navbar-expand` class on the navbar. Link: <https://getbootstrap.com/docs/4.0/components/navbar/>

3. Add a view corresponding to the `Index` action in the `Admin` controller.

```

@model IEnumerable<Product>

@{
 ViewBag.Title = "Admin";
 Layout = "~/Views/Shared/_AdminLayout.cshtml";
}

<h1>Products</h1>

<a asp-action="Create" class="btn btn-primary mb-3">Add Product

<table class="table table-striped table-bordered table-sm">
 <tr>
 <th class="text-right">ID</th>
 <th>Name</th>
 <th class="text-right">Price</th>
 <th class="text-center">Actions</th>
 </tr>
 @foreach (var item in Model)

```

```

{
 <tr>
 <td class="text-right">@item.ProductID</td>
 <td>@item.Name</td>
 <td class="text-right">@item.Price</td>
 <td class="text-center">
 <a asp-action="Edit" class="btn btn-sm btn-warning"
 asp-route-productId="@item.ProductID">
 Edit

 <form
 asp-action="Delete"
 method="post" style="display: inline">
 <input type="hidden" name="ProductId" value="@item.ProductID" />
 <button type="submit" class="btn btn-danger btn-sm">
 Delete
 </button>
 </form>
 </td>
 </tr>
}
</table>

```

## 4. Editing the Products

4. Add an Edit action on the AdminController

```

public IActionResult Edit(int productId)
{
 var product = repository.Products.FirstOrDefault(p => p.ProductID == productId);
 return View(product);
}

```

5. Add the corresponding view. Update the content of the view as follows:

```

@model MVCStore.Models.Product

@{
 ViewBag.Title = "Edit";
 Layout = "_AdminLayout";
}

<h1>Edit product</h1>
<hr />

<form asp-action="Edit">
 <div asp-validation-summary="ModelOnly" class="text-danger"></div>

```

```

<input type="hidden" asp-for="ProductID" />
<div class="form-group">
 <label asp-for="Name" class="control-label"></label>
 <input asp-for="Name" class="form-control" />

</div>
<div class="form-group">
 <label asp-for="Description" class="control-label"></label>
 <input asp-for="Description" class="form-control" />

</div>
<div class="form-group">
 <label asp-for="Price" class="control-label"></label>
 <input asp-for="Price" class="form-control" />

</div>
<div>
 <input type="submit" value="Save" class="btn btn-primary" />
 <a asp-action="Index" class="btn btn-secondary">Back to List
</div>
</form>

```

6. Add a `SaveProduct` method in the `IStoreRepository` interface. “C#  
 public interface `IStoreRepository` { `IEnumerable` `Products` { `get`; }

```

 Task SaveProductAsync(Product product);
} “

```

7. Implement the `SaveProduct` method as follows. C# public  
 async Task `SaveProductAsync`(`Product` `product`) { if  
 (`product.ProductID` == 0) { context.`Products`.Add(`product`);  
 } else { `Product` `dbEntry` = context.`Products`  
 .FirstOrDefault(p => p.`ProductID` == `product.ProductID`);  
 if (`dbEntry` != null) { `dbEntry`.`Name` =  
`product`.`Name`; `dbEntry`.`Description` = `product`.`Description`;  
`dbEntry`.`Price` = `product`.`Price`; } } await  
 context.`SaveChangesAsync`(); }

8. Add the `Edit` action that will handle POST requests on the  
`AdminController`

```

[HttpPost]
public async Task<IActionResult> Edit(Product product)
{
 if (ModelState.IsValid)
 {
 await repository.SaveProductAsync(product);
 TempData["message"] = $"{product.Name} has been saved";
 }
}

```

```

 return RedirectToAction("Index");
 }
 else
 {
 // there is something wrong with the data values
 return View(product);
 }
}

```

Notice the TempData object

9. Update the \_AdminLayout.cshtml layout file in order to display the confirmation message.

```

@if (TempData["message"] != null)
{
 <div class="alert alert-success">@TempData["message"]</div>
}

```

10. Update the Product class as follows.

```

public class Product {
 public int ProductID { get; set; }
 [Required(ErrorMessage = "Please enter a product name")]
 public string Name { get; set; }
 [Required(ErrorMessage = "Please enter a description")]
 public string Description { get; set; }
 [Required]
 [Range(0.01, double.MaxValue, ErrorMessage = "Please enter a positive price")]
 [Column(TypeName = "decimal(8, 2)")]
 public decimal Price { get; set; }
 public string Category { get; set; }
}

```

## 5. Creating New Products

11. Add a Create action to the AdminController class.

```

public IActionResult Create(){
 return View("Edit", new Product());
}

```

## 6. Deleting Products

12. Add a DeleteProduct method to the IStoreRepository interface.

```

Task<Product> DeleteProductAsync(int productID);

```

13. Implement the method in the EFProductRepository class.

```

public async Task<Product> DeleteProductAsync(int productID) {
 Product dbEntry = context.Products
 .FirstOrDefault(p => p.ProductID == productID);

 if (dbEntry != null) {
 context.Products.Remove(dbEntry);
 await context.SaveChangesAsync();
 }

 return dbEntry;
}

```

14. Add the corresponding action to the AdminController

```

[HttpPost]
public async Task<IActionResult> Delete(int productId) {
 Product deletedProduct = await repository.DeleteProductAsync(productId);
 if (deletedProduct != null) {
 TempData["message"] = $"{deletedProduct.Name} was deleted";
 }
 return RedirectToAction("Index");
}

```

15. Add a new unit test class in the MVCStore.Tests project called AdminControllerTests. Add the following test method.

```

[Fact]
public void Can_Delete_Valid_Products() {
 // Arrange - create a Product
 Product prod = new Product { ProductID = 2, Name = "Test" };
 // Arrange - create the mock repository
 Mock<IStoreRepository> mock = new Mock<IStoreRepository>();
 mock.Setup(m => m.Products).Returns(new Product[] {
 new Product {ProductID = 1, Name = "P1"},
 prod,
 new Product {ProductID = 3, Name = "P3"},
 }).AsQueryable<Product>();
 // Arrange - create the controller
 AdminController target = new AdminController(mock.Object);
 // Act - delete the product
 target.Delete(prod.ProductID);
 // Assert - ensure that the repository delete method was
 // called with the correct Product
 mock.Verify(m => m.DeleteProductAsync(prod.ProductID));
}

```

## 7. Bibliography

## MVCStore - Security

- 1. Objectives
- 2. Changing the Context Class
- 3. Creating and Applying the Database Migration
- 4. Defining the Seed Data
- 5. Applying a Basic Authorization Policy
- 6. Bibliography

### 1. Objectives

- implementing the identity framework;

### 2. Changing the Context Class

1. Install the NuGet package “Microsoft.AspNetCore.Identity.EntityFrameworkCore”.
2. Change the `ApplicationDbContext` class as follows.

```
public class ApplicationDbContext : IdentityDbContext<IdentityUser>
{
 public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options): base(options)
 {
 }

 public DbSet<Product> Products { get; set; }
}
```

The `ApplicationDbContext` class is derived from `IdentityDbContext`, which provides Identity-specific features for Entity Framework Core. For the type parameter, we used the `IdentityUser` class, which is the built-in class used to represent users.

3. Install the package “Microsoft.AspNetCore.Identity.UI”. In the `Program` class make the following changes in the `Main` method.

```
public class Program
{
 public static void Main(string[] args)
 {
 var builder = WebApplication.CreateBuilder(args);

 // Add services to the container.
 builder.Services.AddDbContext<ApplicationDbContext>(opts => {
 opts.UseSqlServer(
 builder.Configuration["ConnectionStrings:DefaultConnection"]);
 });
 builder.Services.AddDatabaseDeveloperPageExceptionFilter();

 // !!!! new/updated code {
 builder.Services.AddDefaultIdentity<IdentityUser>(options => options.SignIn.RequireConfirmedAccount = true);
 }
}
```

```

 .AddEntityFrameworkStores<ApplicationDbContext>();
 //}
 builder.Services.AddControllersWithViews();

 var app = builder.Build();

 // Configure the HTTP request pipeline.
 if (app.Environment.IsDevelopment())
 {
 app.UseMigrationsEndPoint();
 }
 else
 {
 app.UseExceptionHandler("/Home/Error");
 // The default HSTS value is 30 days. You may want to change this for production
 app.UseHsts();
 }

 app.UseHttpsRedirection();
 app.UseStaticFiles();

 app.UseRouting();

 // !!!! new/updated code {
 app.UseAuthentication();
 app.UseAuthorization();
 //}

 app.MapControllerRoute("pagination",
 "Products/Page{productPage}",
 new { Controller = "Home", action = "Index" });
 //}
 app.MapDefaultControllerRoute();
 // !!!! new/updated code {
 app.MapRazorPages();
 //}

 app.Run();
}
}

```

The `AddDefaultIdentity` method adds a set of common identity services to the application, including a default UI, token providers, and configures authentication to use identity cookies. Further reading: <https://docs.microsoft.com/en-us/dotnet/api/microsoft.extensions.dependencyinjection.identityservicecollectionuiextensions.adddefaultidentity>

### 3. Creating and Applying the Database Migration

1. Add a new migration to our application by running the following command.

```
Add-Migration Identity
```

2. Apply the migration by calling the following command.

```
Update-Database
```

### 4. Defining the Seed Data

1. Add a new class called `SeedDataIdentity` to the `Data` folder

```
public static class SeedDataIdentity
{
 private const string adminEmail = "admin@test.com";
 private const string adminPassword = "Secret123$";
 public static async Task EnsurePopulatedAsync(IApplicationBuilder app)
 {
 var serviceProvider = app.ApplicationServices
 .CreateScope().ServiceProvider;

 using (var userManager = serviceProvider
 .GetRequiredService<UserManager<IdentityUser>>())
 {
 IdentityUser user = await userManager.FindByEmailAsync(adminEmail);

 if (user == null)
 {
 user = new IdentityUser { UserName = adminEmail, Email = adminEmail };
 await userManager.CreateAsync(user, adminPassword);
 }
 }
 }
}
```

2. Call the `EnsurePopulated` method in the `Configure` method of the `Startup` class.

```
Task.Run(async () =>
{
 await SeedDataIdentity.EnsurePopulatedAsync(app);
}).Wait();
```

**\*Question** - What changes should be made to the application in order to store additional details for a User? > Further reading: <https://docs.microsoft.com/en-us/aspnet/core/security/authentication/add-user-data>

## 5. Applying a Basic Authorization Policy

1. The `[Authorize]` attribute is used to restrict access to action methods or controllers. Decorate the `AdminController` with this attribute.
2. Run the application and try to access any action on the `AdminController`
3. Scaffold Register, Login, and LogOut

Futher reading: <https://docs.microsoft.com/en-us/aspnet/core/security/authentication/identity>

Make sure to have the following code in the selected layout view

```
@RenderSection("Scripts", required: false)
```

4. Check the code in the Register, Login, and LogOut pages. Change their default layout to `_AdminLayout` by modifying in the folder `Areas/Indentity/Pages` the `_ViewStart.cshtml`.
5. Notice that a file called `_LoginPartial.cshtml` has been added to the `Views/Shared` folder.
6. Include the `LoginPartial` in the `_AdminLayout` file

```
<nav class="navbar bg-light navbar-expand-sm">
 <div class="container-fluid">
 MVStore
 <!-- !!!! new/updated code { -->
 <div class="d-sm-inline-flex justify-content-between">
 <partial name="_LoginPartial" />
 </div>
 <!-- } -->
 </div>
</nav>
```

`d-inline-flex` creates an inline flexbox container: <https://getbootstrap.com/docs/5.2/utilities/display/>  
For `sm` check <https://getbootstrap.com/docs/5.2/layout/breakpoints/#available-breakpoints> .

## 6. Bibliography

## MVCStore - Authorization, Roles

- 1. Objectives
- 2. Documentation
- 3. Introduction
- 4. Roles
- 5. Bibliography

### 1. Objectives

### 2. Documentation

- Authorization Overview: <https://docs.microsoft.com/en-us/aspnet/core/security/authorization/introduction>

### 3. Introduction

**Authorization** refers to the process that determines what a user is able to do. For example, an administrative user is allowed to create a document library, add documents, edit documents, and delete them. A non-administrative user working with the library is only authorized to read the documents.

**Authorization** is orthogonal and independent from authentication. However, authorization requires an authentication mechanism. Authentication is the process of ascertaining who a user is. Authentication may create one or more identities for the current user.

### 4. Roles

1. Update the `EnsurePopulatedAsync` method in the `SeedDataIdentity` class to also create a role.

```
var roleManager = serviceProvider.GetRequiredService<RoleManager<IdentityRole>>();

var roleName = "ProductManagement";

if (!await roleManager.RoleExistsAsync(roleName))
 await roleManager.CreateAsync(new IdentityRole(roleName));
```

2. Add a user with the “ProductManagement” role

```
var adminWithRoleEmail = "adminRole@test.com";
var adminWithRolePassword = "Secret123$";

IdentityUser adminWithRole = await userManager.FindByEmailAsync(adminWithRoleEmail);
if (adminWithRole == null)
{
 adminWithRole = new IdentityUser { UserName = adminWithRoleEmail, Email = adminWithRoleEmail };
 await userManager.CreateAsync(adminWithRole, adminWithRolePassword);
}
```

```
 await userManager.AddToRoleAsync(adminWithRole, roleName);
 }
}
```

3. Modify the Main method in the Program class as follows

```
// !!!! new/updated code {
/*builder.Services.AddDefaultIdentity<IdentityUser>(options => options.SignIn.RequireCo
 .AddEntityFrameworkStores<ApplicationDbContext>());*/

builder.Services.AddIdentity<IdentityUser, IdentityRole>()
 .AddRoleManager<RoleManager<IdentityRole>>()
 .AddDefaultUI()
 .AddEntityFrameworkStores<ApplicationDbContext>();
//}
```

4. Update the last few lines in the Index.cshtml corresponding to the AdminController as follows.

```
<td class="text-center">
 <a asp-action="Edit" class="btn btn-sm btn-warning"
 asp-route-productId="@item.ProductID">
 Edit

 @if (User.IsInRole("ProductManagement"))
 {
 <form
 asp-action="Delete"
 method="post" style="display: inline">
 <input type="hidden" name="ProductId" value="@item.ProductID" />
 <button type="submit" class="btn btn-danger btn-sm">
 Delete
 </button>
 </form>
 }
</td>
```

5. You should also decorate the actions that will only be available to users that have the ProductManagement role as follows.

```
[Authorize(Roles = "ProductManagement")]
```

## 5. Bibliography

# Cross-Site Request Forgery (XSRF/CSRF) Attacks

- 1. Objectives
- 2. Documentation
- 3. Step 1 - Create an application with individual user accounts
- 4. Step 2 - Add the CSRF vulnerable webpage
- 5. Malicious or infected web site
- 6. Attacks
- 7. Bibliography

## 1. Objectives

## 2. Documentation

Cross-site request forgery (also known as XSRF or CSRF, pronounced see-surf) is an attack against web-hosted apps whereby a malicious web app can influence the interaction between a client browser and a web app that trusts that browser. These attacks are possible because web browsers send some types of authentication tokens automatically with every request to a website. This form of exploit is also known as a one-click attack or session riding because the attack takes advantage of the user's previously authenticated session.

Recommended further reading: Open Web Application Security Project - OWASP (description of the vulnerability in general): [https://www.owasp.org/index.php/Cross-Site\\_Request\\_Forgery\\_\(CSRF\)](https://www.owasp.org/index.php/Cross-Site_Request_Forgery_(CSRF)); CSRF in ASP.NET: <https://docs.microsoft.com/en-us/aspnet/core/security/anti-request-forgery>

## Vulnerable website - IronBank Web'Banking

### 3. Step 1 - Create an application with individual user accounts

:octocat: Full source code available, check the 10 - Vulnerabilities - CSRF (Cross-Site Request Forgery) folder.

1. Create a new "ASP.NET Core Web App (Model-View-Controller)" project named "IronBankWeb".
2. For "Authentication type" choose "Individual Accounts".
3. Notice that in the "Data" folder the application already includes the necessary database migrations.
4. Create an SQL Server database (such as "ism-csrf") and update the connection string in `appsettings.json`.

5. Run the application.
6. Choose the “Register” link and create a new account.

#### 4. Step 2 - Add the CSRF vulnerable webpage

1. Add a new controller, called `BankAccountController`.
2. Decorate the controller using the `[Authorize]` annotation in order to prevent any unauthenticated requests towards the actions on this controller.
3. Add an action called `Transfer` to the `BankAccountController` as follows.

```
public IActionResult Transfer()
{
 return View();
}
```

4. Add a new viewmodel class to the `Models` folder.

```
public class TransferViewModel
{
 [Required]
 public required string DestinationAccount { get; set; }
 [Required]
 public decimal? Amount { get; set; }
}
```

5. Add a new View for the `Transfer` action.

```
@model TransferViewModel
@{
 ViewData["Title"] = "Transfer";
}

<h2>Transfer</h2>

@if(TempData["message"] != null)
{
 <div class="alert alert-danger">
 @TempData["message"]
 </div>
}

<form method="post">
 <label asp-for="DestinationAccount"></label>
 <input asp-for="DestinationAccount" />
 <label asp-for="Amount"></label>
 <input asp-for="Amount" />
```

```

 <input type="submit" />
 </form>

```

6. Add a POST method for the form in the View

```

[HttpPost]
public IActionResult Transfer(TransferViewModel transfer)
{
 if (ModelState.IsValid)
 {
 TempData["Message"] = $"You have transfered {transfer.Amount} euros to {transfer.DestinationAccount}";
 return RedirectToAction(nameof(Transfer));
 }
 return View(transfer);
}

```

## 5. Malicious or infected web site

:octocat: Full source code available, check the 10 - Vulnerabilities  
- CSRF (Cross-Site Request Forgery) folder.

1. Create an html file with the following content.

```

<!DOCTYPE html>
<html>
<head>
 <title>Malicious or infected web site</title>
</head>
<body style="text-align:center;">

 <h1>Congratulations!</h1>
 <h1>You have won a Samsung Galaxy Fold (2.000 euros)!!!!</h1>
 <form action="http://good-banking-site.com/api/account" method="post">
 <input type="hidden" name="DestinationAccount" value="R085INGBXXXXXXXXXX">
 <input type="hidden" name="Amount" value="1000000">
 <input type="submit" value="Click to collect your prize!">
 </form>
</body>
</html>

```

2. You will need to replace “http://good-banking-site.com/api/account” with the actual address of the vulnerable form (ex: https://localhost:5001/BankAccount/Transfer).

## 6. Attacks

1. Try to perform the attack. You will notice that in modern browsers the attack is not working.
2. Check the cookie policy in Google Chrome.. Notice that the `AspNetCore.Identity.Application` cookie has the `SameSite` policy set to `Strict` (or `Lax`). Let's change the `SameSite` policy to `SameSiteMode.None` by adding the following code to the `Main` method of the `Program` class.

Documentation for `SameSite`: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Set-Cookie#samesitesamesite-value>

Potentially valid reasons for setting the `SameSite` policy to `SameSiteMode.None`: <https://andrewlock.net/understanding-samesite-cookies/#:~:text=The%20one%20advantage%20of%20SameSite,and%20Lax%20won't%20wo>

```
builder.Services.ConfigureApplicationCookie(options =>
{
 options.Cookie.SameSite = SameSiteMode.None;
});
```

3. Try to perform the attack again. The attack should work successfully.
4. Let's prevent the attack by adding the `[ValidateAntiForgeryToken]` annotation to the `Transfer` action, as shown below.

```
[HttpPost]
[ValidateAntiForgeryToken]
public IActionResult Transfer(TransferViewModel transfer)
{

}
```

5. Check the hidden input in the form and the cookies set by the application.

Have you noticed that the token in the cookie and the one in the hidden input are different? You can read more about the reason at: <https://stackoverflow.com/questions/20911470/why-is-there-a-difference-in-the-validateantiforgerytoken-cookie-value-and-hidde>

## 7. Bibliography

- Prevent Cross-Site Request Forgery (XSRF/CSRF) attacks in ASP.NET Core: <https://docs.microsoft.com/en-us/aspnet/core/security/anti-request-forgery?>
- OWASP CSRF: [https://www.owasp.org/index.php/Cross-Site\\_Request\\_Forgery\\_\(CSRF\)](https://www.owasp.org/index.php/Cross-Site_Request_Forgery_(CSRF))

# Vulnerabilities - CORS (Cross-Origin Resource Sharing) Attacks

- 1. Objectives
- 2. Documentation
- 3. Scenario
- 4. Step 1 - Create the first application (the one containing the secret)
- 5. Step 2 - Create the second application (that is vulnerable to XSS)
- 6. Step 3 - Performing the attack
- 7. Bibliography

## 1. Objectives

## 2. Documentation

You can learn more details about this type of attack at <https://portswigger.net/web-security/cors> .

## 3. Scenario

In the following we want to focus on “Exploiting XSS via CORS trust relationships”

Even “correctly” configured CORS establishes a trust relationship between two origins. If a website trusts an origin that is vulnerable to Cross-Site Scripting (XSS), then an attacker could exploit the XSS to inject some JavaScript that uses CORS to retrieve sensitive information from the site that trusts the vulnerable application.

Source: <https://portswigger.net/web-security/cors> .

Let’s suppose that we have two applications. The first one includes a webpage where users can find some secret information (ex: their API Key, serial number, etc.) after authenticating themselves. The second application could be a partner website or another website belonging to our organization that needs to be able to perform client-side requests to the first application.

Let’s suppose that the attacker has already been able to perform a successful **XSS** attack on a **Vulnerable** website . He could now include the following JavaScript that will be executed for each user that access the **Partner** website.

We should specify in the fetch method that the credentials should be sent when making the request to the **Attacked** website. Documentation: <https://developer.mozilla.org/en-US/docs/Web/API/fetch#credentials>

```
fetch('https://localhost:7093/BankAccount/Transfer', { credentials: "include" })
 .then((response) => response.text())
 .then(html => console.log(html)) // here we could send the data
```

#### 4. Step 1 - Create the first application (the one containing the secret)

1. In an existing or a new application implementing individual user accounts add the following `APITokensController` controller:

```
[Authorize]
public class APITokensController : Controller
{
 public IActionResult Index()
 {
 return View();
 }
}
```

2. Corresponding to the `Index` action, add the following view:

```
@{
 ViewData["Title"] = "Authentication Tokens";
}
```

```
<h2>Authentication Tokens</h2>
```

```
Access Token: BKBEHGIE87274234293
```

#### 5. Step 2 - Create the second application (that is vulnerable to XSS)

3. Create an application including a page (`html`, `Razor`, etc.) that contains the following JavaScript code (let's suppose that it is there due to an **XSS** attack). Note: Don't forget to update the port.

```
fetch('https://localhost:7093/APITokens', { credentials: "include" })
 .then((response) => response.text())
 .then(html => console.log(html)) // here we could send the data
```

#### 6. Step 3 - Performing the attack

1. Check what gets displayed in the console for the second application.
2. Update the `Main` method in the `Program` class as follows for the first application to enable **CORS**. > Note: Don't forget to update the port. You should run the application using a webserver (e.g., "Live Server" in VS Code, etc.)

```
First, call AddCors. string corsPolicyForSecondDomain = "AllowSecondDomain";
builder.Services.AddCors(options => { options.AddPolicy(name:
corsPolicyForSecondDomain, policy
=> { policy.WithOrigins("https://
```

```

.AllowAnyMethod()
.AllowCredentials();
call UseCors.

app.UseRouting();

app.UseCors(corsPolicyForSecondDomain);

app.UseAuthentication();

```

3. Check what gets displayed in the console for the second application.

## 7. Bibliography

- Fetch method: <https://developer.mozilla.org/en-US/docs/Web/API/fetch#credentials>
- ASP.NET CORS settings: <https://learn.microsoft.com/en-us/aspnet/core/security>
- <https://lo-victoria.com/introduction-to-cross-origin-resource-sharing-cors>
- <https://portswigger.net/web-security/cors>
- <https://medium.com/@zhaojunemail/sop-cors-csrf-and-xss-simply-explained-with-examples-af6119156726>

## APIs with ASP.NET Core

- 1. Objectives
- 2. Documentation
- 3. Activity
- 4. Bibliography

## APIs with ASP.NET Core

### 1. Objectives

### 2. Documentation

<https://dotnet.microsoft.com/en-us/apps/aspnet/apis> <https://learn.microsoft.com/en-us/aspnet/core/fundamentals/apis>

### 3. Activity

Follow the tutorial at <https://learn.microsoft.com/en-us/aspnet/core/tutorials/first-web-api>

### 4. Bibliography



**Department of Economic Informatics and Cybernetics**  
Bucharest University of Economic Studies

# Web and Cloud Security

ASP.NET – Part I



# ASP.NET MVC Core, C# Language



C#

- url: <https://docs.microsoft.com/en-us/aspnet/>

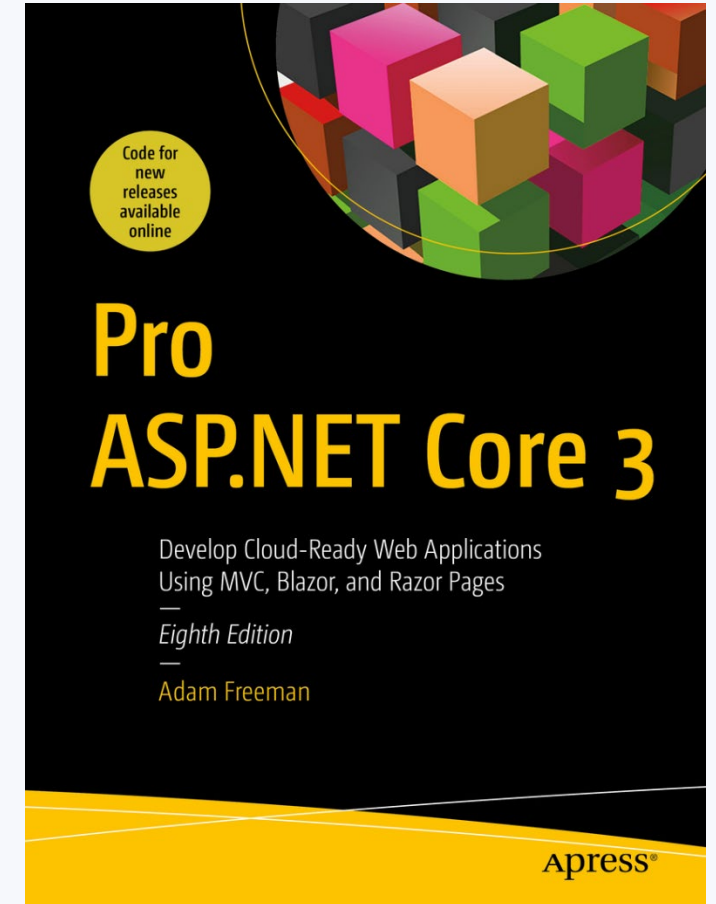
# About me



<https://ro.linkedin.com/in/cotfasliviu>

# Recommended Reading / Watching

- Pro ASP.NET Core 3
  - ADAM FREEMAN
  - Now in its 8th edition, the best-selling book on MVC is now updated for ASP.NET Core.
  - <https://www.apress.com/gp/book/9781484254394>
  - <https://github.com/Apress/pro-asp.net-core-3>



## Recommended Reading / Watching

- Free Books:
  - <https://www.syncfusion.com/succinctly-free-ebooks?category=all&searchkey=asp.net&type=all>



## Further Reading / Watching

- Courses on Microsoft Learn - <https://docs.microsoft.com/en-us/learn/>
  - Free
- Courses on PluralSight - [www.pluralsight.com](http://www.pluralsight.com)
  - Free trial (10 days)
  - Free **limited** access through <https://azure.microsoft.com/en-us/free/students/>

# API reference and Source code

- Documentation:
  - <https://docs.microsoft.com/en-us/aspnet/>
- API reference:
  - <https://docs.microsoft.com/en-us/dotnet/api>
- Source code:
  - <https://github.com/aspnet/Home>
- Various samples:
  - <https://code.msdn.microsoft.com/>

# Contents

Brief History of the Microsoft Web Stack

HelloWorld (ASP .NET)

Key Benefits of ASPNET Core

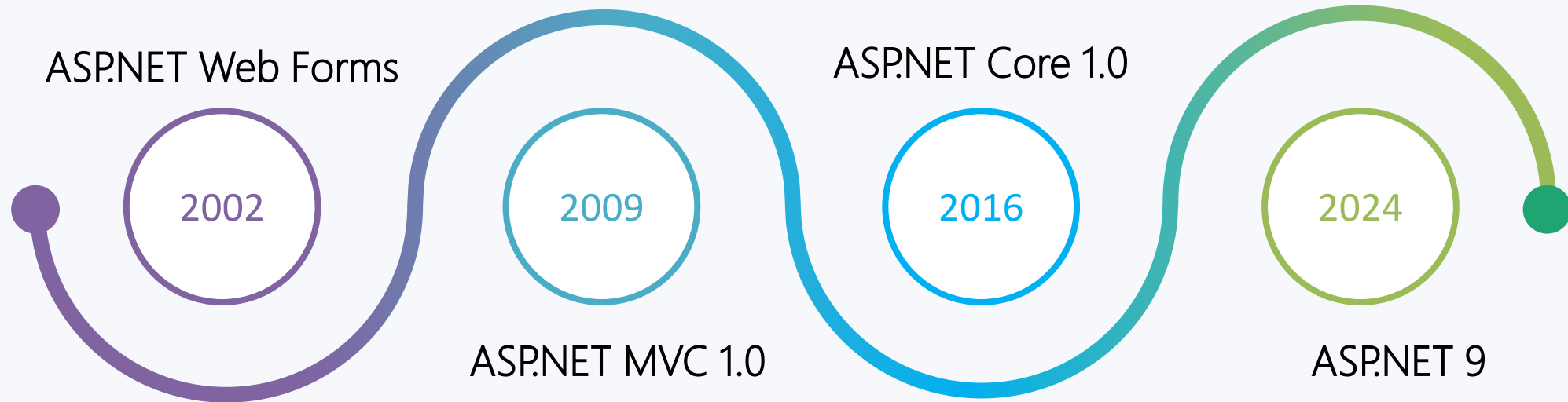
Model-View-Controller Pattern

Project Structure

Thank you!

# Brief History of the Microsoft Web Stack

# History



# ASP.NET Web Forms

- Launched in 2002;
- Shortcomings **ASP.NET Web Forms** :
  - **View State weight**: The actual mechanism for maintaining state across requests resulted in large blocks of data being transferred between the client and server. This data could reach hundreds of kilobytes in even modest web applications, and it went back and forth with every request, leading to slower response times and increasing the bandwidth demands of the server.
  - **Page life cycle**: The mechanism for connecting client-side events with server-side event handler code, part of the page life cycle, could be complicated and delicate. Few developers had success manipulating the control hierarchy at runtime without creating View State errors or finding that some event handlers mysteriously fail to execute.

## ASP.NET Web Forms

- **Limited control over HTML:** Server controls rendered themselves as HTML, but not necessarily the HTML you wanted. In early versions of ASP.NET, the HTML output failed to meet with web standards or make good use of Cascading Style Sheets (CSS), and server controls generated unpredictable and complex ID attributes that are hard to access using JavaScript. These problems have improved in recent Web Forms releases, but it can still be tricky to get the HTML you expect.
- **Low testability :** The designers of Web Forms could not have anticipated that automated testing would become an essential component of software development. The tightly coupled architecture they designed was unsuitable for unit testing. Integration testing could be a challenge, too.

## ASP.NET MVC

- Launched in 2007 (CTP), 2009 (1.0);
- Based on the **Model-View-Controller** (MVC) pattern to keep a clear separation between business logic and presentation logic, allowing complete control over the HTML markup.
- Released as a separate library, one not included in the framework. Thanks to this release model, and not relying 100% on the IDE for the design of the UI, it was possible to easily update it, keeping it more in line with the fast-paced world of web development.

## ASP.NET MVC

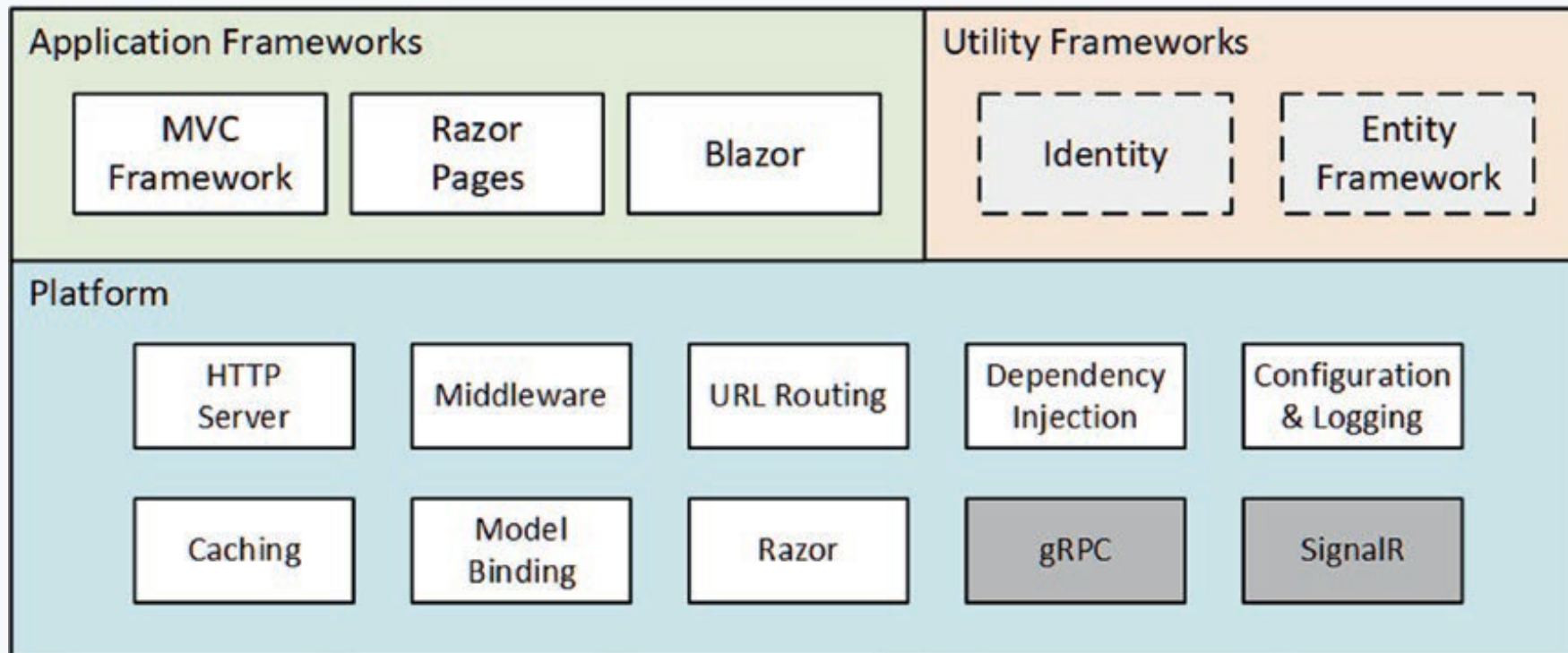
- Although solving the problem of the slow-release cycle and removing the HTML markup abstraction, ASP.NET MVC still suffered from:
  - One couldn't run .NET applications on a non-Windows system.
- The dependency on the full framework made the .NET apps less suitable for high-density scenarios, like the cloud, where hundreds of applications run on a single machine and must scale up very fast.
- The complexity of the .NET project system prevented the development of .NET apps outside of Visual Studio.

## ASP.NET Core

- Launched in 2016.
- Built on **.NET Core**, which is a **cross-platform** version of the .NET Framework without the Windows-specific application programming interfaces (APIs).
- Possible for developers to create ASP.NET Core web applications on Linux and OS X/macOS.
- ASP.NET Core is a completely new framework. It is simpler, it is easier to work with, and it is free of the legacy that comes from Web Forms. And, since it is based on .NET Core, it supports the development of web applications on a range of platforms and containers.

# ASP.NET Core

- provides the functionality of the original ASP.NET MVC Framework and includes the functionality that was previously provided by Web API.

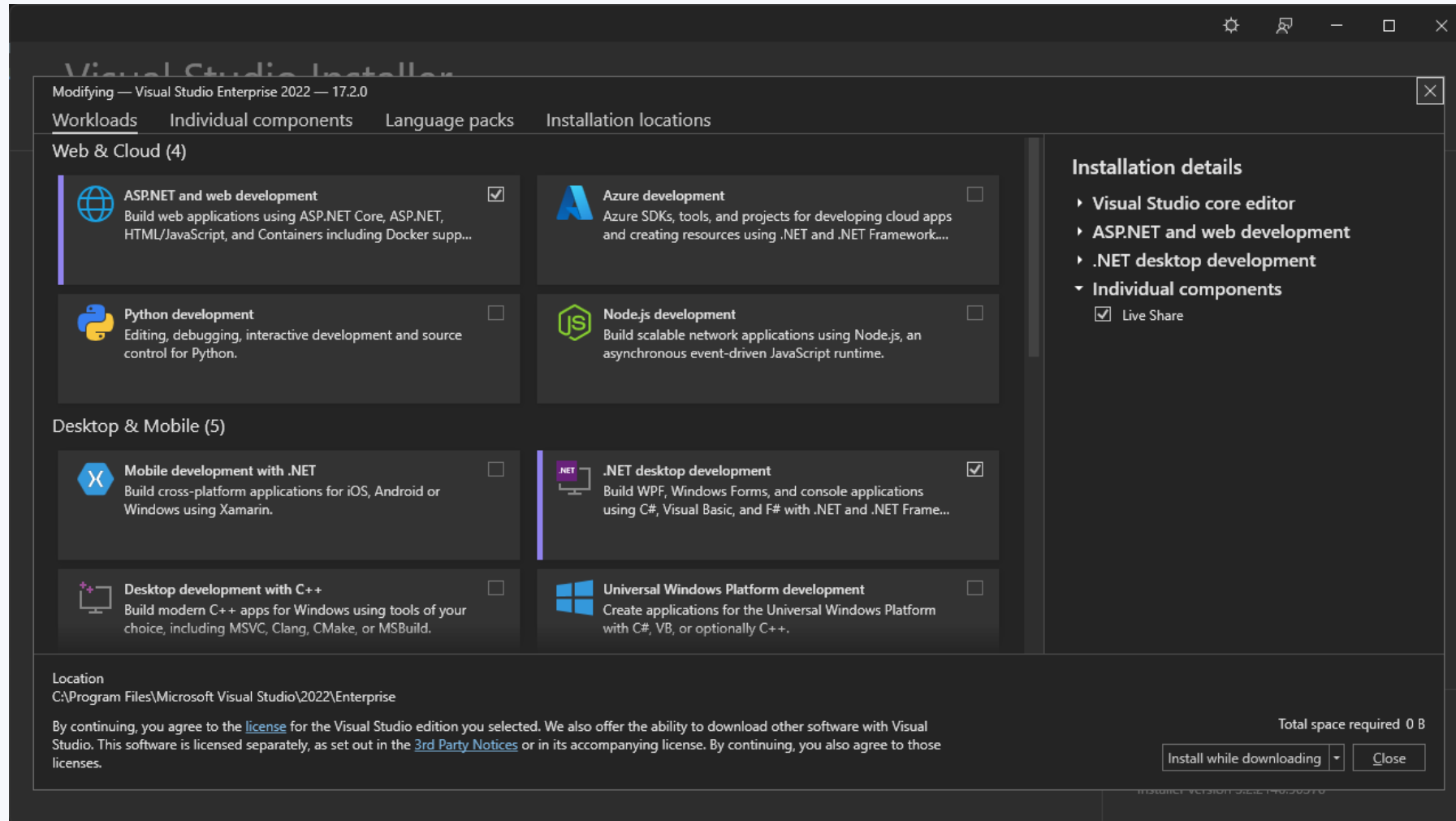


HelloWorld (ASP .NET)

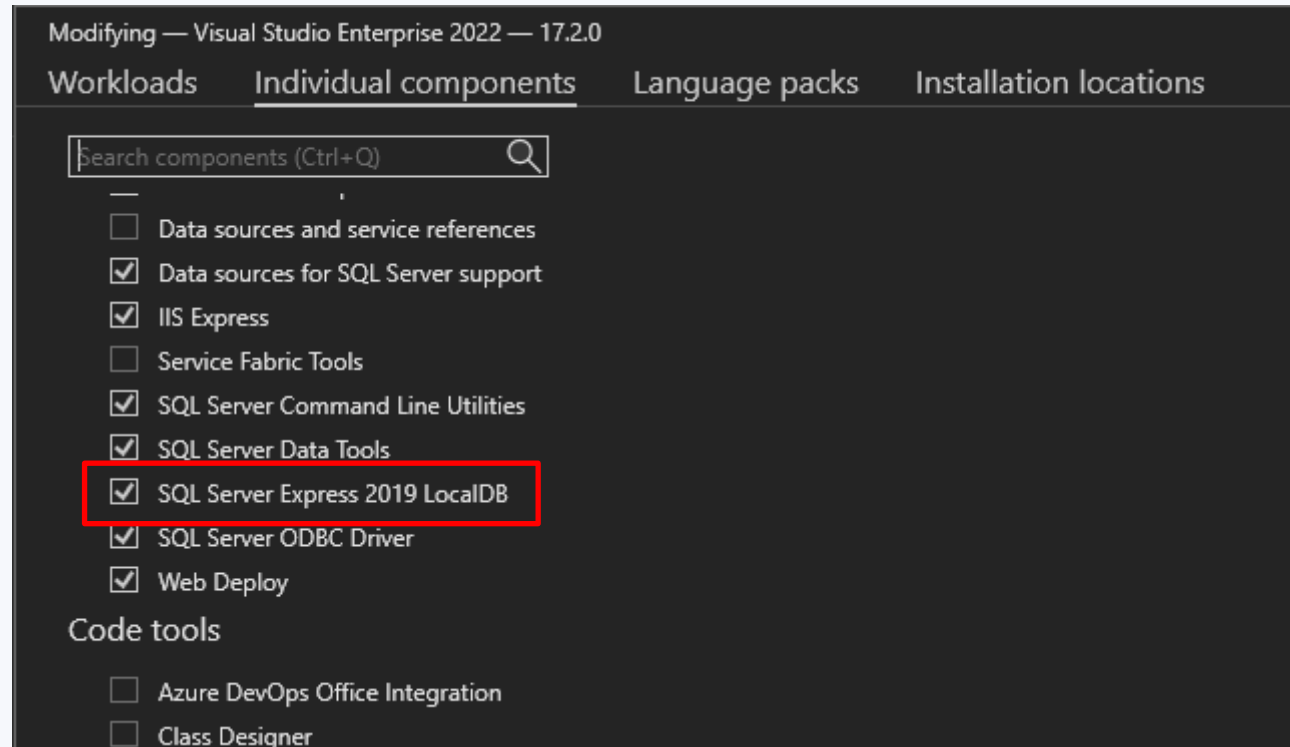
# Installing Visual Studio 2022

- Visual Studio 2022 is available in Community, Professional and Enterprise editions.
- **Community** is available for free and has all of the features required for day-to-day development. Download and run the installer from [www.visualstudio.com](https://www.visualstudio.com)
- **Professional** is available through [azure.microsoft.com/en-us/free/students/](https://azure.microsoft.com/en-us/free/students/)
- Make sure that the “ASP.NET and web development” workload is installed, either during setup or afterwards from Tools > Get Tools and Features.

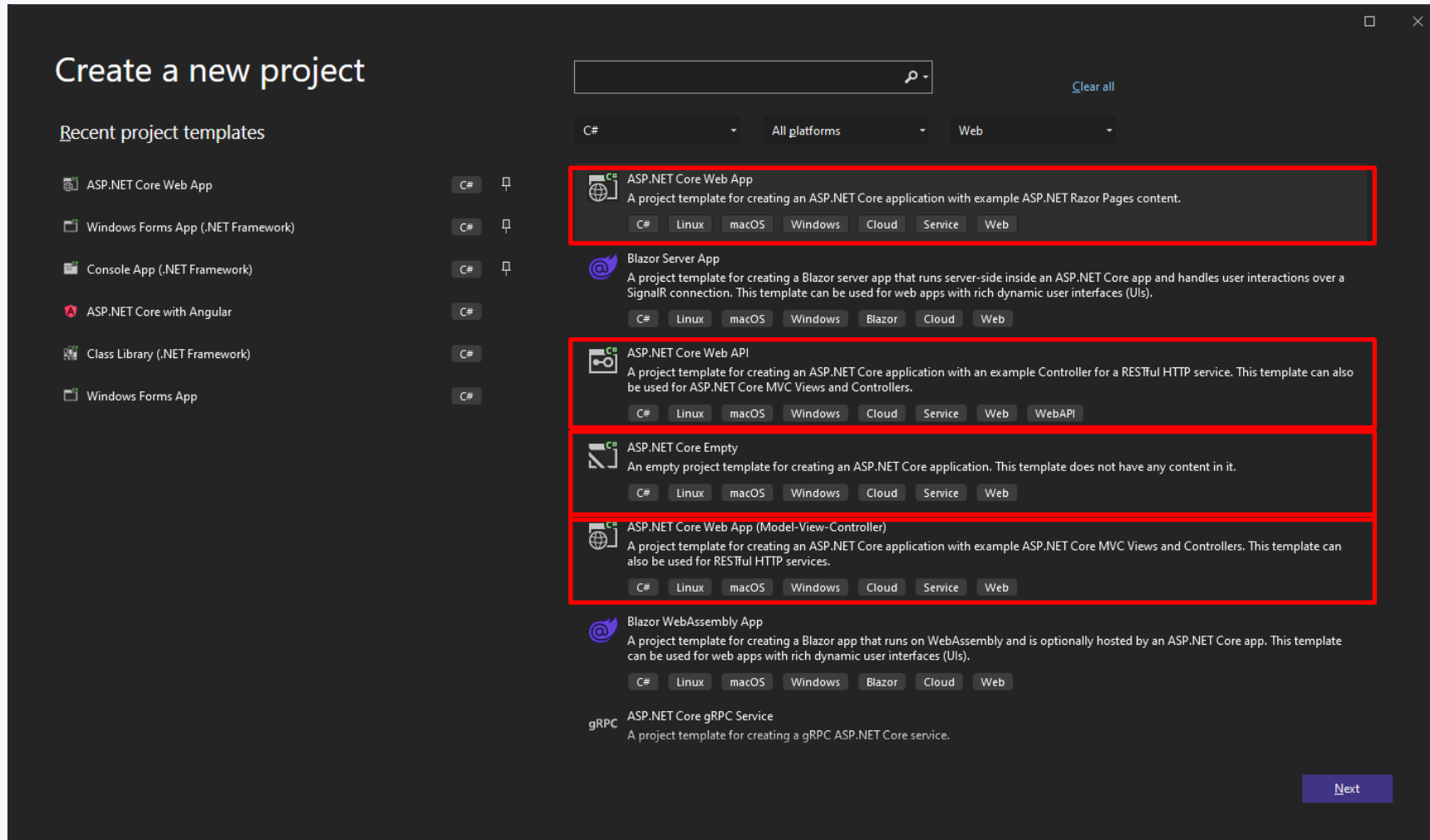
# Installing Visual Studio 2022



# Installing Visual Studio 2022



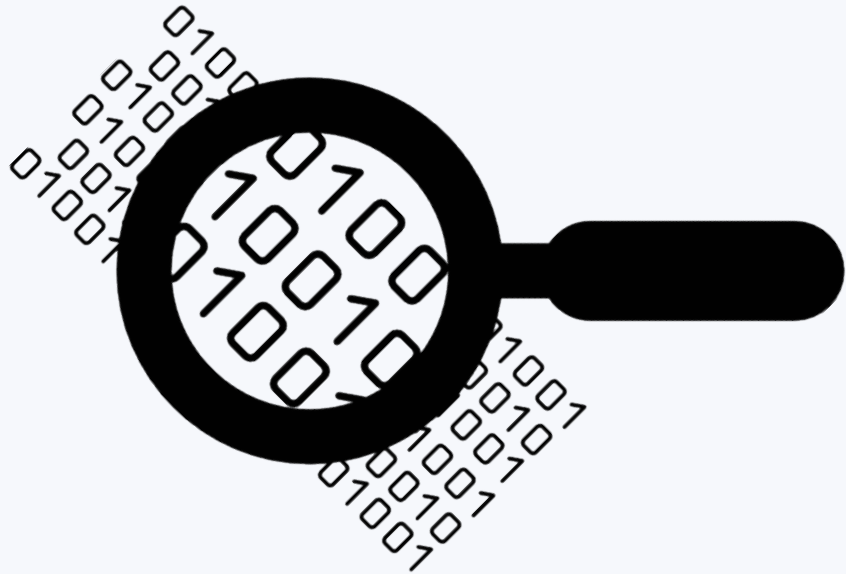
# New Project



# C# Language

- C# (pronounced "C sharp") is a general-purpose, **strongly typed**, **type-safe** language, **object-oriented** programming language, designed for building a variety of applications.
- Focused on developer **productivity**.
- C# syntax is based on the C & C++ syntax.
- Platform-neutral, but it was written to work well with the Microsoft .NET Framework.

# Demo

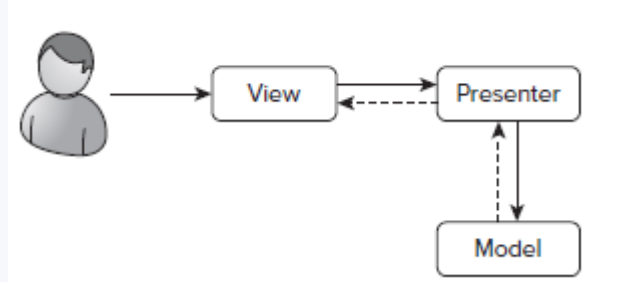


HelloWorld

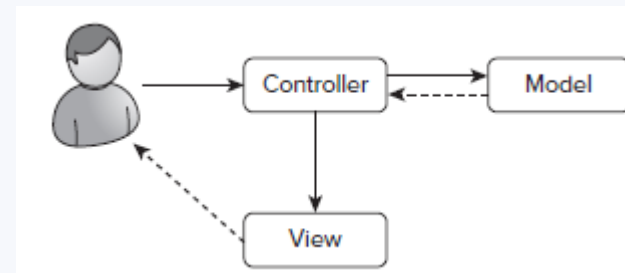
# Model-View-Controller Pattern

# MVC Pattern

- The Model-View-Controller Pattern separates an application into three main components: the **model**, the **view**, and the **controller**. The controller interacts with the model based on the requirements of the request. It renders the appropriate view and supplies it with the necessary data to display.



**MVP**



**MVC**

- It's a **User Interface Pattern**, therefore one of the many patterns you will use in a complex project.

# MVC Pattern - Components

- The **model** which contain or represent the data that users work with.
- **Views** define how the application UI will be displayed.
- **Controllers** process incoming requests, perform operations on the model and select views to render to the user.

# MVC Pattern

- Each piece of the MVC architecture is well-defined and self-contained, which is referred to as the *separation of concerns* .
- The logic that manipulates the data in the model is contained *only* in the model;
- The logic that displays data is *only* in the view
- The code that handles user requests and input is contained *only* in the controller.
- With a clear division between each of the pieces, your application will be easier to maintain and extend over its lifetime, no matter how large it becomes.

# Models

- Models—the M in MVC - contain the data that users work with.
- There are two broad types of model:
  - **view models (presentation models)** - represent just data passed from the controller to the view,
  - **domain models** - which contain the data in a business domain, along with the operations, transformations, and rules for creating, storing, and manipulating that data, collectively referred to as the model logic .

Models are the definition of the universe your application works in.

# Models

- In a banking application, for example, the model represents everything in the bank that the application supports, such as accounts, the general ledger, and credit limits for customers, as well as the operations that can be used to manipulate the data in the model, such as depositing funds and making withdrawals from the accounts.
- The model is also responsible for preserving the overall state and consistency of the data—for example, making sure that all transactions are added to the ledger and that a client doesn't withdraw more money than he is entitled to or more money than the bank has.

# Models

The model in an application built using the MVC pattern *should*

- Contain the domain data
- Contain the logic for creating, managing, and modifying the domain data
- Provide a clean API that exposes the model data and operations on it

The model *should not*

- Expose details of how the model data is obtained or managed (in other words, details of the data storage mechanism should not be exposed to controllers and views)
- Contain logic that transforms the model based on user interaction (because that is the controller's job)
- Contain logic for displaying data to the user (that is the view's job)

# Models

- The benefits of ensuring that the model is isolated from the controller and views are that you can test your logic more easily and that enhancing and maintaining the overall application is simpler and easier.
- **Tip** Many developers new to the MVC pattern get confused with the idea of including logic in the data model, believing that the goal of the MVC pattern is to separate data from logic. This is a misapprehension: the goal of the MVC pattern is to divide an application into three functional areas, each of which may contain both logic *and* data. **The goal isn't to eliminate logic from the model. Rather, it is to ensure that the model only contains logic for creating and managing the model data.**

# Controllers

- Controllers are the connective tissue in the MVC pattern, acting as conduits between the data model and views. **Controllers** define **actions** that provide the business logic that operates on the data model and that provide the data that views display to the user.
- A controller built using the MVC pattern *should*
  - Contain the actions required to update the model based on user interaction
- The controller *should not*
  - Contain logic that manages the appearance of data (that is the job of the view)
  - Contain logic that manages the persistence of data (that is the job of the model)

# Views

- Views contain the logic required to display data to the user or to capture data from the user so that it can be processed by a controller action.
- Views *should*:
  - Contain the logic and markup required to present data to the user
- Views *should not*
  - Contain complex logic (this is better placed in a controller)
  - Contain logic that creates, stores, or manipulates the domain model

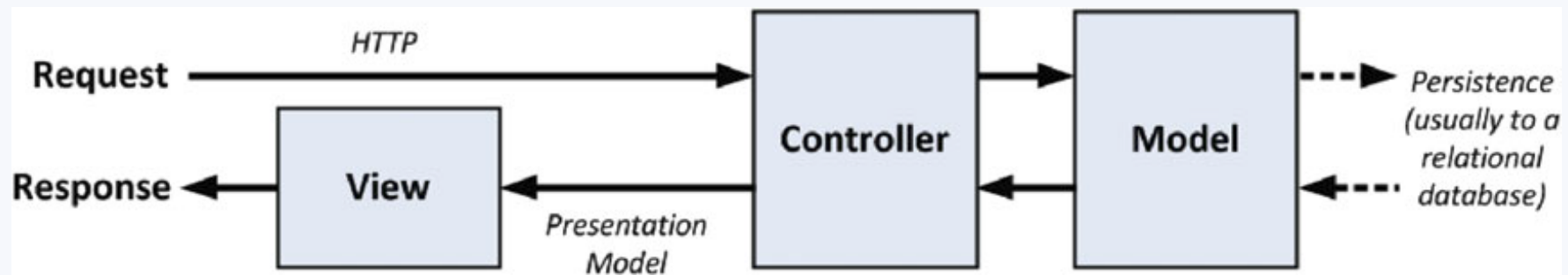
# Views

- Views *can* contain logic, but it should be simple and used sparingly. Putting anything but the simplest method calls or expressions in a view makes the overall application harder to test and maintain

# ASP.NET Core MVC

- ASP.NET MVC is a framework for building web applications that applies the general Model View Controller pattern to the ASP.NET framework.
- In ASP.NET Core MVC, controllers are C# classes, usually derived from the `Microsoft.AspNetCore.Mvc.Controller` class.
- Each public method in a class derived from Controller is an *action method*, which is associated with a URL.
- When a request is sent to the URL associated with an action method, the statements in that action method are executed in order to perform some operation on the domain model and then to select a view to display to the client.

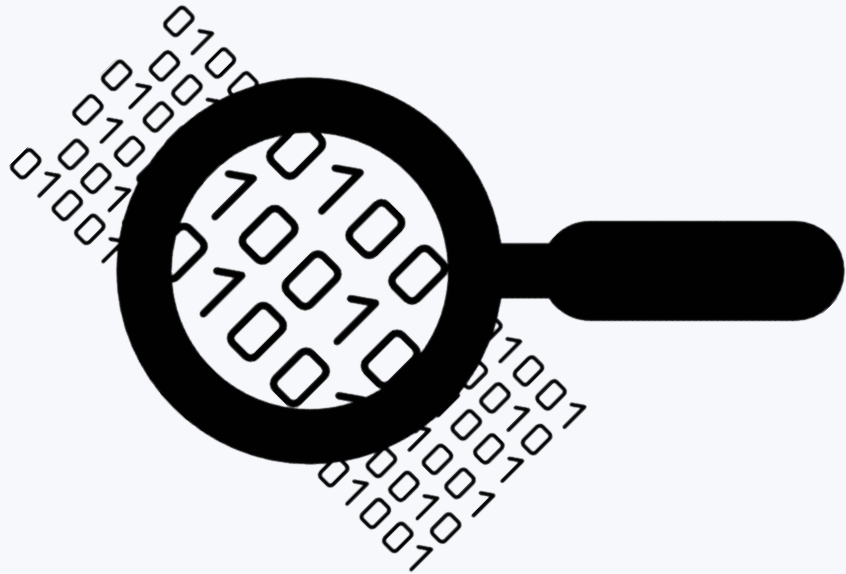
# ASP.NET Core MVC



# ASP.NET Core MVC

- ASP.NET Core MVC uses a *view engine* , known as Razor, which is the component responsible for processing a view in order to generate a response for the browser. Razor views are HTML templates that contain C# logic that is used to process model data to generate dynamic content that responds to changes in the model.
- ASP.NET Core MVC doesn't apply any constraints on the implementation of your domain model. You can create a model using regular C# objects and implement persistence using any of the databases, object relational mapping frameworks, or other data tools supported by .NET.

# Demo



CourseInvites

# Key Benefits of ASP.NET Core

## MVC Architecture

- The ASP.NET MVC architecture gives you a great start in making your application maintainable and testable because you naturally separate different application concerns into independent pieces.

# Extensibility

- ASP.NET Core is built as a series of independent components that have well-defined characteristics, satisfy a .NET **interface** or that are built on an **abstract base class**. Thus, it is possible to replace key components with ones of your own implementation.
- In general, the ASP.NET Core MVC gives you these three options for each component:
  - Use the default implementation of the component as it stands (which should be enough for most applications).
  - Derive a subclass of the default implementation to tweak its behavior.
  - Replace the component entirely with a new implementation of the interface or abstract base class.

## Tight Control over HTML and HTTP

- ASP.NET produces clean, standards-compliant markup. Its built-in **tag helpers** produce standards-compliant output
- ASP.NET encourages you to craft simple, elegant markup styled with CSS.
- ASP.NET makes it easy to use client-side libraries such as jQuery, Angular, or the Bootstrap CSS library.
- ASP.NET works in tune with HTTP. You have control over the requests passing between the browser and server, so you can fine-tune your user experience. Ajax is made easy and creating web services to receive browser HTTP requests is a simple process.

## Testability

- Each piece of the ASP.NET platform and the ASP.NET MVC framework can be isolated and replaced for unit testing, which can be performed using any popular open source testing framework, such as xUnit.
- ASP.NET applications also work well with **UI automation** testing tools, too. You can write test scripts that simulate user interactions without needing to guess which HTML element structures, CSS classes, or IDs the framework will generate, and you do not have to worry about the structure changing unexpectedly.

## Powerful Routing System

- The style of uniform resource locators (URLs) has evolved as web application technology has improved.
- URLs like this one:

`/App_v2/User/Page.aspx?action=show%20prop&prop_id=82742`

are increasingly rare, replaced with a simpler, cleaner format like this:

`/to-rent/chicago/2303-silver-street`

# Reasons for caring about the structure of URLs

- First, search engines give weight to keywords found in a URL. A search for “rent in Chicago” is much more likely to turn up the simpler URL.
- Second, many web users are now savvy enough to understand a URL and appreciate the option of navigating by typing it into their browser’s address bar.
- Third, when someone understands the structure of a URL, they are more likely to link to it, share it.
- Fourth, it doesn’t expose the technical details, folder, and file name structure of your application to the public Internet, so you are free to change the underlying implementation without breaking all your incoming links.

# Powerful Routing System

- Clean URLs were hard to implement in earlier frameworks, but ASP.NET Core MVC uses a feature known as *URL routing* to provide clean URLs by default.
- This gives you control over your URL schema and its relationship to your application, offering you the freedom to create a pattern of URLs that is meaningful and useful to your users, without the need to conform to a predefined pattern.

# Modern API

- Microsoft's .NET platform has evolved with each major release, supporting—and even defining—the state-of-the art aspects of modern programming.
- ASP.NET Core MVC is built for .NET Core, so its API can take full advantage of language and runtime innovations familiar to C# programmers, including the **async/await** keyword, extension methods, lambda expressions, anonymous and dynamic types, and Language Integrated Query (LINQ).
- Many of the ASP.NET Core MVC API methods and coding patterns follow a cleaner, more expressive composition than was possible with earlier platforms.

# Cross-Platform

- Previous versions of ASP.NET were specific to Windows, requiring a Windows desktop to write web applications and a Windows server to deploy and run them. Microsoft made ASP.NET Core cross-platform, both for development and for deployment. .NET Core is available for different platforms—including Linux and OS X/macOS—and is likely to be ported to others.
- ASP.NET applications can be developed in Visual Studio Code, which is cross-platform, which means that ASP.NET development is no longer restricted to Windows.

# Open Source

- Unlike previous Microsoft web development platforms, you are free to download the source code for ASP.NET Core and ASP.NET Core MVC and even modify and compile your own version of it.
- This is invaluable when your debugging trail leads into a system component and you want to step into its code (and even read the original programmers' comments). It is also useful if you are building an advanced component and want to see what development possibilities exist or how the built-in components actually work.
- You can download the ASP.NET Core and ASP.NET Core MVC source code from <https://github.com/aspnet> .

# Project Structure

# Visual Studio Project Templates

- Visual Studio offers several ASP.NET Core templates:
  - Empty
  - Web API
  - Web App
  - Web App (Model-View-Controller)
- Different authentication methods can be chosen.

# Visual Studio Project Templates

- The **Empty** project template contains the plumbing for ASP.NET Core but doesn't include the libraries or configuration required for an MVC application.
- The **Web API** project template includes ASP.NET Core and MVC, with a sample application that demonstrates how to receive and process Ajax requests from clients.
- The **Web App** project template is preconfigured for using Razor Pages.
- The **Web App** (Model-View-Controller) project template is preconfigured for developing applications using the Model-View-Controller pattern.

# Visual Studio Project Templates

- The templates are just different starting points into the same functionality, and you can add whatever functionality you need to projects created with any of the templates.
- So, the real difference between the project templates is the initial set of libraries, configuration files, code, and content that Visual Studio adds when it creates the project.
- Regardless of the template that you use to create a project, there are some common folders and files that will appear. Some of the items in a project have special roles that are hard-coded into ASP.NET or MVC or one of the tools that Visual Studio provides support for. Others are subject to naming conventions that are used in most ASP.NET or MVC projects.

# Visual Studio Project Structure

Folder or File	Description
/Areas	Areas are a way of partitioning a large application into smaller pieces.
/Dependencies	The Dependencies item provides details of all the packages a project relies on.
/Components	This is where view component classes, which are used to display self contained features such as shopping carts, are defined.
/Controllers	This is where you put your controller classes. This is a convention. You can put your controller classes anywhere you like, because they are all compiled into the same assembly.

# Visual Studio Project Structure

Folder or File	Description
/Data	This is where database context classes should be defined, but they are also frequently defined in the Models folder.
/Migrations	This is where details of database schemas are stored so that deployment databases can be updated.
/Models	This is where you put your view model and domain model classes. This is a convention. You can define your model classes anywhere in the project or in a separate project.

# Visual Studio Project Structure

Folder or File	Description
/Views	This directory holds views and partial views, usually grouped together in folders named after the controller with which they are associated.
/Views/Shared	This directory holds layouts and views that are not specific to a single controller.

# Visual Studio Project Structure

Folder or File	Description
/Views/_ViewImports.cshtml	This file is used to specify the namespaces that will be included in Razor view files.
/Views/_ViewStart.cshtml	This file is used to specify a default layout for the Razor view engine.
/Program.cs	This class configures the hosting platform for the application

# Visual Studio Project Structure

Folder or File	Description
/Startup.cs	This class configures the application
/wwwroot	This is where you put static content such as CSS files and images. It is also where the package manager installs JavaScript and CSS packages

# Convention Over Configuration

- There are two kinds of conventions in an MVC project:
  - The first kind is just suggestions as to how you might like to structure your project. For example, it is conventional to put the third-party JavaScript and CSS packages you rely on in the `wwwroot/lib` folder. This is where other MVC developers would expect to find them and where the package manager will install them. But you are free to rename the `lib` folder, or remove it entirely and put your packages somewhere else. That would not prevent MVC from running your application as long as the script and link elements in your views refer to the location you settle on.

# Convention Over Configuration

- The other kind of convention arises from the principle of *convention over configuration* , which was one of the main selling points that made Ruby on Rails so popular. Convention over configuration means that you don't need to explicitly configure associations between controllers and their views, for example.
- You just follow a certain naming convention for your files, and everything just works. There is less flexibility in changing your project structure when dealing with this kind of convention.

# Convention Over Configuration

- By following a certain naming convention, everything “works”.
- Principles
  - avoid configuring things that can be inferred;
  - all convention-based defaults can be overridden.
- Example:
  - No need to explicitly configure associations between **controllers** and their **views**;

# Convention Over Configuration

- Three core directions:
  - Controllers;
  - Models;
  - Views.
- Following common conventions and using a software design pattern facilitates code understanding even in large applications.

# Controller Conventions

- All controller classes end in "Controller" (ex: HomeController");
- Controllers are referenced only by the first part in their names;
- New controller conventions can be created by implementing IControllerFactory.

# View Conventions

- Views and partial views should be located in the folder `/Views/ControllerName`.
  - `ex:` `/Views/Home` for the `HomeController`
- The default view for an action should be named after that method.
  - `ex:` for an action `Index`, the view should be named `Index.cshtml`
- Choosing the right view:
  - The default view will be used for: `return View();`
  - A different view can be specified by name: `return View("Other");`

# Layout Conventions

- Layout file names are prefixed with “\_” (underscore);
- Layouts are placed in the folder `/Views/Shared/`;
- The default layout can be changed in `_ViewStart.cshtml`;
- Specific layouts can be defined for every view:

**ex:** `@{Layout = “~Views/Shared/_OtherLayout.cshtml”; }`

- Layouts can be disabled for a given view:

**ex:** `@{Layout = null;}`

Thank you!



**Department of Economic Informatics and Cybernetics**  
Bucharest University of Economic Studies

# Web and Cloud Security

ASP.NET MVC Core 2 – Part II



# ASP.NET MVC Core, C# Language



C#

- url: <https://docs.microsoft.com/en-us/aspnet/>

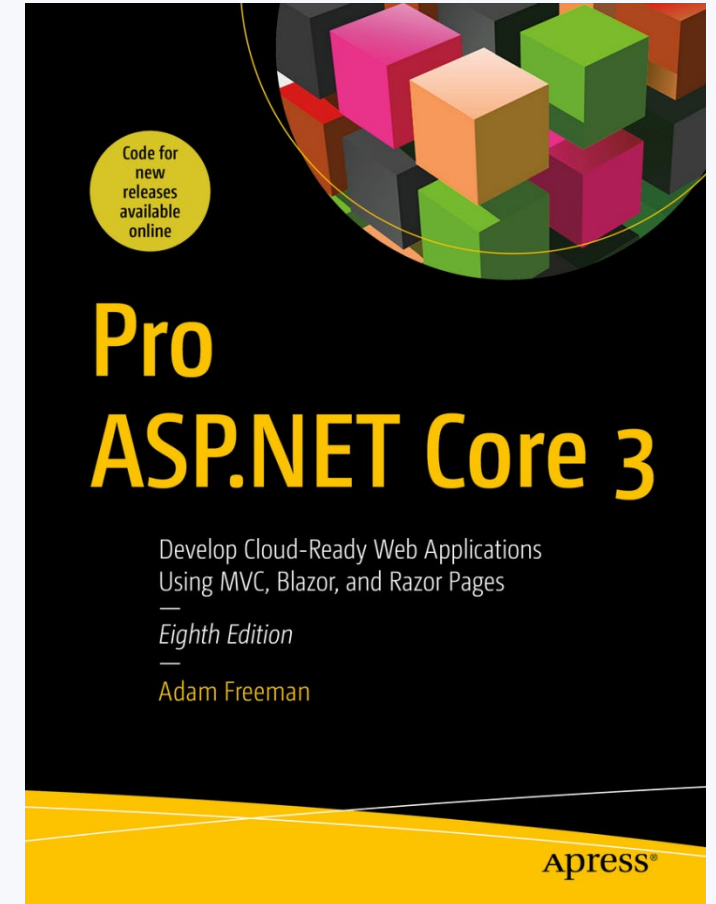
# About me



<https://ro.linkedin.com/in/cotfasliviu>

# Recommended Reading / Watching

- Pro ASP.NET Core 3
  - ADAM FREEMAN
  - Now in its 8th edition, the best-selling book on MVC is now updated for ASP.NET Core.
  - <https://www.apress.com/gp/book/9781484254394>
  - <https://github.com/Apress/pro-asp.net-core-3>



## Recommended Reading / Watching

- Free Books:
  - <https://www.syncfusion.com/succinctly-free-ebooks?category=all&searchkey=asp.net&type=all>



## Further Reading / Watching

- Courses on Microsoft Learn - <https://docs.microsoft.com/en-us/learn/>
  - Free
- Courses on PluralSight - [www.pluralsight.com](http://www.pluralsight.com)
  - Free trial (10 days)
  - Free **limited** access through <https://azure.microsoft.com/en-us/free/students/>

# API reference and Source code

- Documentation:
  - <https://docs.microsoft.com/en-us/aspnet/>
- API reference:
  - <https://docs.microsoft.com/en-us/dotnet/api/?view=net-5.0>
- Source code:
  - <https://github.com/aspnet/Home>
- Various samples:
  - <https://code.msdn.microsoft.com/>

# Contents

Authentication

Authorization

Enforcing SSL

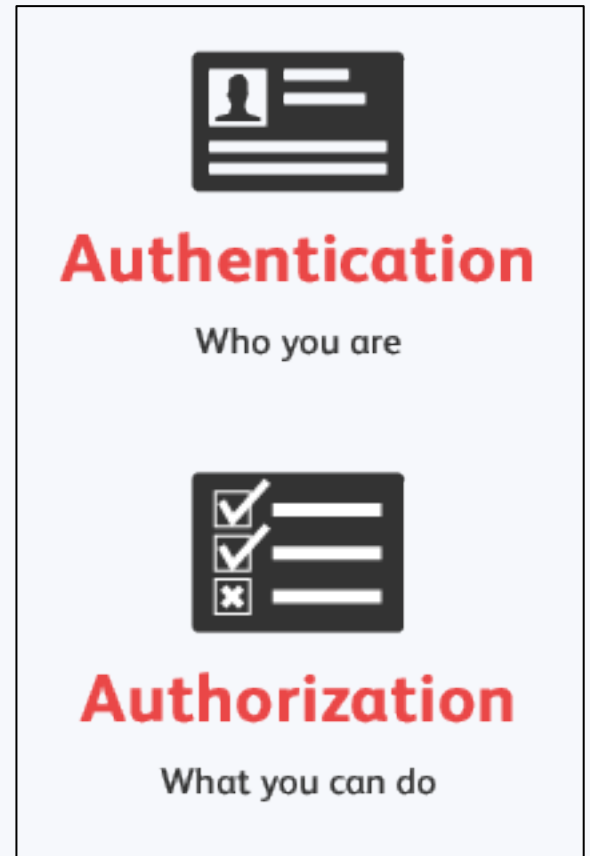
# Security in ASP.NET Core

- ASP.NET Core enables developers to easily configure and manage security for their apps. ASP.NET Core contains features for managing authentication, authorization, data protection, SSL enforcement, app secrets, anti-request forgery protection, and CORS management. These security features allow you to build robust yet secure ASP.NET Core apps.
- Further reading: <https://docs.microsoft.com/en-us/aspnet/core/security>

# Authentication

# Authentication

- Authentication - determining the identity of a user.
- Authorization - determining whether a user is allowed to perform an action



# ASP.NET Core Identity

- ASP.NET Core Identity is an API from Microsoft to manage users in ASP.NET applications
- Users can create an account and login with their Active Directory account, with a user name and password or they can use an external login providers such as Facebook, Google, Microsoft Account, Twitter and more
- ASP.NET Core Identity can be configured to use a SQL Server database to store user names, passwords, and profile data. Alternatively, you can use your own persistent store to store data in another persistent storage, such as Azure Table Storage
- OSS alternatives are also available: [link](#)

# ASP.NET Core Identity

- Why is it useful?
  - User management is an important feature for most applications, and ASP.NET Core Identity provides a ready-made and well-tested platform that doesn't require you to create custom versions of commonly demanded functions.
- NuGet Package:
  - `Microsoft.AspNetCore.Identity.EntityFrameworkCore`
- Documentation: [link](#)

# IdentityUser Class

<u>AccessFailedCount</u>	Gets or sets the number of failures for the purposes of logout.(Inherited from <u>IdentityUser(Of TKey, TLogin, TRole, TClaim).</u> )
<u>Email</u>	Gets or sets the email for the user.(Inherited from <u>IdentityUser(Of TKey, TLogin, TRole, TClaim).</u> )
<u>EmailConfirmed</u>	Gets or sets a value that indicates whether the email is confirmed.(Inherited from <u>IdentityUser(Of TKey, TLogin, TRole, TClaim).</u> )
<u>Id</u>	Gets or sets the user identifier.(Inherited from <u>IdentityUser(Of TKey, TLogin, TRole, TClaim).</u> )
<u>LockoutEnabled</u>	Gets or sets a value that indicates whether logout enabled for this user.(Inherited from <u>IdentityUser(Of TKey, TLogin, TRole, TClaim).</u> )
<u>LockoutEndDateUtc</u>	Gets or sets the date time value (in UTC) when logout ends, any time in the past is considered not locked out.(Inherited from <u>IdentityUser(Of TKey, TLogin, TRole, TClaim).</u> )
<u>Logins</u>	Gets the collection of logins for the user.(Inherited from <u>IdentityUser(Of TKey, TLogin, TRole, TClaim).</u> )

# IdentityUser Class

<u>PasswordHash</u>	Gets or sets the salted/hashed form of the user password.(Inherited from <u>IdentityUser(Of TKey, TLogin, TRole, TClaim).</u> )
<u>PhoneNumber</u>	Gets or sets the phone number for the user.(Inherited from <u>IdentityUser(Of TKey, TLogin, TRole, TClaim).</u> )
<u>PhoneNumberConfirmed</u>	Gets or sets the value that indicates whether the phone number is confirmed. The default is false.(Inherited from <u>IdentityUser(Of TKey, TLogin, TRole, TClaim).</u> )
<u>Roles</u>	Gets the collection of roles for the user.(Inherited from <u>IdentityUser(Of TKey, TLogin, TRole, TClaim).</u> )
<u>SecurityStamp</u>	Gets or sets a random value that changes when a user's credentials change.(Inherited from <u>IdentityUser(Of TKey, TLogin, TRole, TClaim).</u> )
<u>TwoFactorEnabled</u>	Gets or sets a value that indicates whether two-factor authentication is enabled for the user.(Inherited from <u>IdentityUser(Of TKey, TLogin, TRole, TClaim).</u> )
<u>UserName</u>	Gets or sets the user name.(Inherited from <u>IdentityUser(Of TKey, TLogin, TRole, TClaim).</u> )

# ASP.NET Core Identity

- The identity services should be added to the application in the ConfigureServices method in the Startup class:

```
public void ConfigureServices(IServiceCollection services)
{
 services.AddDbContext<ApplicationDbContext>(options =>
 options.UseSqlServer(
 Configuration["Data:Database:ConnectionString"]));
 services.AddTransient<IProductRepository, EFProductRepository>();

 services.AddIdentity<IdentityUser, IdentityRole>()
 .AddEntityFrameworkStores<ApplicationDbContext>();

 services.AddMvc();
}
```

# Configure Identity

- Password policy:

```
options.Password.RequireDigit = true;
options.Password.RequiredLength = 8;
options.Password.RequireNonAlphanumeric = false;
options.Password.RequireUppercase = true;
options.Password.RequireLowercase = false;
```

- User's lockout:

```
// Lockout settings
options.Lockout.DefaultLockoutTimeSpan =
 TimeSpan.FromMinutes(30);
options.Lockout.MaxFailedAccessAttempts = 10;
options.Lockout.AllowedForNewUsers = true;
```

# Configure Identity

- Application's cookie settings:

```
// Cookie settings
options.Cookies.ApplicationCookie.CookieName = "YouAppCookieName";
options.Cookies.ApplicationCookie.ExpireTimeSpan = TimeSpan.FromDays(150);
options.Cookies.ApplicationCookie.LoginPath = "/Account/LogIn";
options.Cookies.ApplicationCookie.LogoutPath = "/Account/LogOff";
options.Cookies.ApplicationCookie.AccessDeniedPath = "/Account/AccessDenied";
options.Cookies.ApplicationCookieAutomaticAuthenticate = true;
options.Cookies.ApplicationCookie.AuthenticationScheme =
Microsoft.AspNetCore.Authentication.Cookies.CookieAuthenticationDefaults.AuthenticationScheme;
options.Cookies.ApplicationCookie.ReturnUrlParameter =
Microsoft.AspNetCore.Authentication.Cookies.CookieAuthenticationDefaults.ReturnUrlParameter;
```

# Authentication using Facebook, Google

- Enabling users to sign in with their existing credentials is convenient for the users and shifts many of the complexities of managing the sign-in process onto a third party.
- How to implement: [link](#)
- Introduction to OAuth 2: [link](#)



# Demo

<https://github.com/liviucotfas/ase-web-and-cloud-applications-security/blob/master/6%20-%20MVCStore%20-%20Security.md>



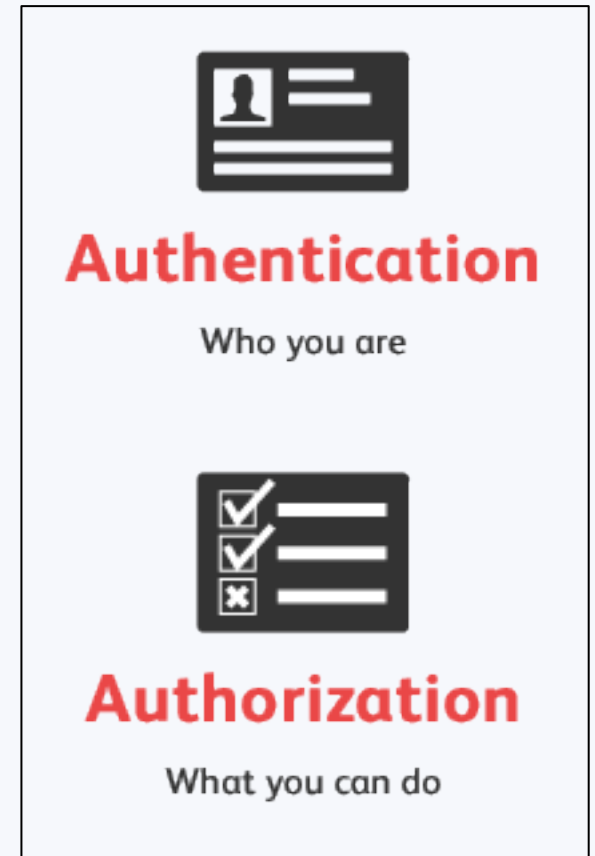
## Recommended reading

- Configure Identity primary keys data type: [link](#)
- Two-factor authentication with SMS: [link](#)
- Azure Active Directory: [link](#)

# Authorization

# Authorization

- **Authorization** - determining whether a user is allowed to perform an action
- Example: an administrative user is allowed to create a document library, add documents, edit documents, and delete them. A non-administrative user working with the library is only authorized to read the documents.
- Authorization is orthogonal and independent from authentication, which is the process of ascertaining who a user is. Authentication may create one or more identities for the current user.



# Authorization

- ASP.NET Core authorization provides a simple declarative **role** and a **policy based model**.
- Authorization is expressed in requirements, and handlers evaluate a user's claims against requirements. Imperative checks can be based on simple policies or policies which evaluate both the user identity and properties of the resource that the user is attempting to access.
- Namespace: Microsoft.AspNetCore.Authorization
- Further reading: [link](#)

## [Authorize]

- [AuthorizeAttribute]
  - tells MVC that only requests from authenticated users should be processed
  - properties: Roles, Policy
  - can be applied both at action and at controller level
- The ASP.NET platform provides information about the user through the HttpContext object, which is used by the Authorize attribute to check the status of the current request and see whether the user has been authenticated. The HttpContext.User property returns an implementation of the IPrincipal interface, which is defined in the System.Security.Principal namespace.

## [Authorize]

- action level

```
public class HomeController : Controller {
 [Authorize]
 public ActionResult Index(){
 return View();
 }
}
```

- controller level

```
[Authorize]
public class HomeController : Controller {
 public ActionResult Index(){
 return View();
 }
}
```

# [Authorize]

## Imperative

```
public ActionResult Details(int id)
{
 if (!User.IsInRole("EmployeeViewer"))
 return new HttpUnauthorizedResult();

 // Action logic
}
```

## Declarative (recommended)

```
[Authorize(Roles = "EmployeeViewer")]
public ActionResult Details(int id)
{
 // Action logic
}
```

# Security Attributes

- [AllowAnonymous]
  - Specifies that actions and controllers are skipped by AuthorizeAttribute during authorization.
  - Example: the login action

```
[Authorize]
public class AccountController : Controller {
 [AllowAnonymous]
 public IActionResult Login() {
 return View();
 }

 [HttpPost]
 [AllowAnonymous]
 [ValidateAntiForgeryToken]
 public async Task<IActionResult> Login(LoginModel details) {
 return View(details);
 }
}
```

Enforcing SSL

## Enforcing SSL

- [RequireHttpsAttribute]
  - can be used to decorate controllers or actions;
- Can also be configured globally:

```
// Requires using Microsoft.AspNetCore.Mvc;
public void ConfigureServices(IServiceCollection services){
 services.Configure<MvcOptions>(options =>
 {
 options.Filters.Add(new RequireHttpsAttribute());
 });
};
```

- **Note:** The highlighted code above requires all requests to use HTTPS, therefore HTTP requests are ignored.

# Enforcing SSL

- Redirects all HTTP requests to HTTPS:

```
public void Configure(IApplicationBuilder app, IHostingEnvironment env)
{

 app.UseHttpsRedirection();
 app.UseStaticFiles();
 app.UseCookiePolicy();

 app.UseMvc();
}
```

Further reading: [docs.microsoft.com/en-us/aspnet/core/security/enforcing-ssl](https://docs.microsoft.com/en-us/aspnet/core/security/enforcing-ssl)

# HTTP Strict Transport Security Protocol (HSTS)

- Is an opt-in security enhancement that's specified by a web app through the use of a response header.
- When a browser that supports HSTS receives this header:
  - The browser stores configuration for the domain that prevents sending any communication over HTTP. The browser forces all communication over HTTPS.
  - The browser prevents the user from using untrusted or invalid certificates. The browser disables prompts that allow a user to temporarily trust such a certificate.
- Further details: [docs.microsoft.com/en-us/aspnet/core/security/enforcing-ssl](https://docs.microsoft.com/en-us/aspnet/core/security/enforcing-ssl)

# HTTP Strict Transport Security Protocol (HSTS)

```
public void Configure(IApplicationBuilder app, IHostingEnvironment env)
{
 if (env.IsDevelopment())
 {
 app.UseDeveloperExceptionPage();
 }
 else
 {
 app.UseExceptionHandler("/Error");
 app.UseHsts();
 }
 ...
}
```

Thank you!



**Department of Economic Informatics and Cybernetics**  
Bucharest University of Economic Studies

# Web and Cloud Security

ASP.NET – Part III



# ASP.NET MVC Core, C# Language



C#

- url: <https://docs.microsoft.com/en-us/aspnet/>

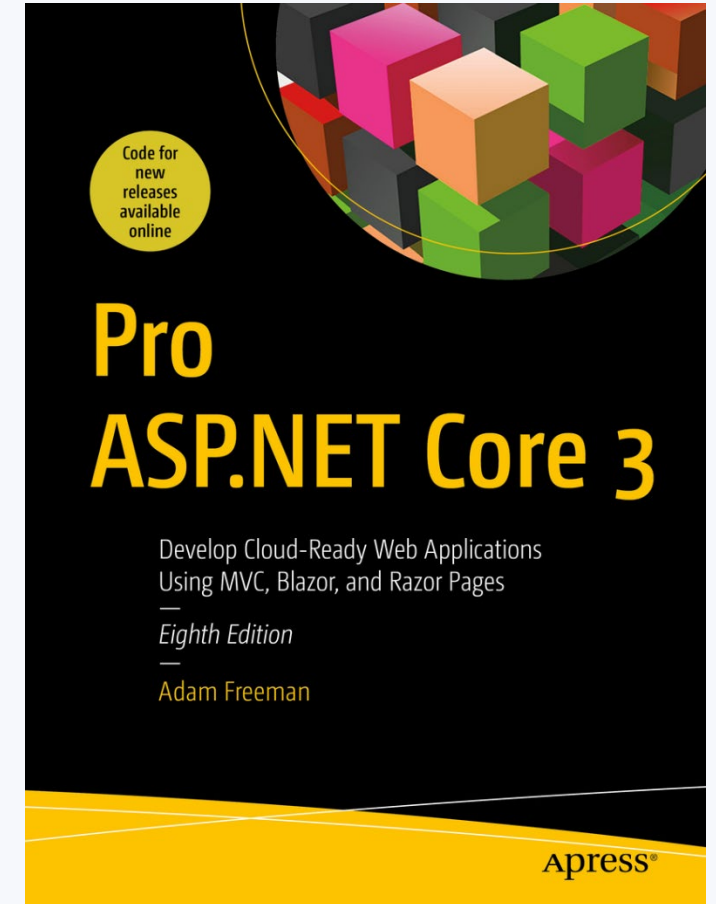
# About me



<https://ro.linkedin.com/in/cotfasliviu>

# Recommended Reading / Watching

- Pro ASP.NET Core 3
  - ADAM FREEMAN
  - Now in its 8th edition, the best-selling book on MVC is now updated for ASP.NET Core.
  - <https://www.apress.com/gp/book/9781484254394>
  - <https://github.com/Apress/pro-asp.net-core-3>



## Recommended Reading / Watching

- Free Books:
  - <https://www.syncfusion.com/succinctly-free-ebooks?category=all&searchkey=asp.net&type=all>



## Further Reading / Watching

- Courses on Microsoft Learn - <https://docs.microsoft.com/en-us/learn/>
  - Free
- Courses on PluralSight - [www.pluralsight.com](http://www.pluralsight.com)
  - Free trial (10 days)
  - Free **limited** access through <https://azure.microsoft.com/en-us/free/students/>

# API reference and Source code

- Documentation:
  - <https://docs.microsoft.com/en-us/aspnet/>
- API reference:
  - <https://docs.microsoft.com/en-us/dotnet/api/?view=net-5.0>
- Source code:
  - <https://github.com/aspnet/Home>
- Various samples:
  - <https://code.msdn.microsoft.com/>

# Contents

Cross-Site Request Forgery (XSRF/CSRF)

Cross-Origin Requests (CORS)

Cross-Site Scripting (XSS)

Security – Other recommendations

# Security in ASP.NET Core

- ASP.NET Core enables developers to easily configure and manage security for their apps. ASP.NET Core contains features for managing authentication, authorization, data protection, SSL enforcement, app secrets, anti-request forgery protection, and CORS management. These security features allow you to build robust yet secure ASP.NET Core apps.
- Further reading: <https://docs.microsoft.com/en-us/aspnet/core/security>

Cross-Site Request Forgery (XSRF/CSRF)

# Cross-Site Request Forgery (XSRF/CSRF)

- Cross-site request forgery (also known as XSRF or CSRF, pronounced see-surf) is an attack against web-hosted applications whereby a malicious web site can influence the interaction between a client browser and a web site that trusts that browser.
- These attacks are made possible because web browsers send some types of authentication tokens automatically with every request to a web site. This form of exploit is also known as a one-click attack or as session riding, because the attack takes advantage of the user's previously authenticated session.
- A malicious site sends POST/GET requests to the attacked site (in which the user is authenticated).

# An example of a CSRF attack

1. A user logs into [www.example.com](http://www.example.com), using forms authentication.
2. The server authenticates the user and issues a response that includes an authentication cookie.
3. The user visits a malicious site.
4. This malicious site contains the following HTML form:

```
<h1>You Are a Winner!</h1>
<form action="http://example.com/api/account" method="post">
<input type="hidden" name="Account" value="R085INGXXX" />
<input type="hidden" name="Amount" value="1000000" />
<input type="submit" value="Click Me"/>
</form>
```

**Note:** the form action posts to the vulnerable site, not to the malicious site. This is the “cross-site” part of CSRF.

# An example of a CSRF attack

1. The user clicks the submit button. The browser automatically includes the authentication cookie for the requested domain (the vulnerable site in this case) with the request.
2. The request runs on the server with the user's authentication context, and can do anything that an authenticated user is allowed to do.

Although this example requires the user to click the form button, the malicious page could just as easily run a script that automatically submits the form or sends a form submission as an AJAX request. The form could also be hidden using CSS so the user never realizes it's present. Moreover, using SSL does not prevent a CSRF attack, because the malicious site can send an `https://` request. Some attacks can target site endpoints that respond to GET requests, in which case even an image tag can be used to perform the action.

## An example of a CSRF attack

- Typically, CSRF attacks are possible against web sites that use cookies for authentication, because browsers send all relevant cookies to the destination web site.
- However, CSRF attacks are not limited to exploiting cookies. For example, Basic and Digest authentication are also vulnerable. After a user logs in with Basic or Digest authentication, the browser automatically sends the credentials until the session ends.
- Users can guard against CSRF vulnerabilities by:
  - Logging off web sites when they have finished using them
  - Clearing their browser's cookies periodically

## How does ASP.NET Core MVC address CSRF?

- The most common approach to defending against CSRF attacks is the Synchronizer Token Pattern (STP).
- STP is a technique used when the user requests a page with form data. The server sends a token associated with the current user's identity to the client.
- The client must send back the token to the server for verification. If the server receives a token that doesn't match the authenticated user's identity, the request should be rejected.
- The token is unique and unpredictable. The token can also be used to ensure proper sequencing of a series of requests (ensuring page 1 precedes page 2 which precedes page 3).

## How does ASP.NET Core MVC address CSRF?

- ASP.NET Core MVC will generate Antiforgery Tokens by default in all the forms:

```
<form method="post">
```

```
</form>
```

```
@using (Html.BeginForm("ChangePassword", "Manage"))
```

```
{
```

```
}
```

## How does ASP.NET Core MVC address CSRF?

- In each of the above cases, ASP.NET Core will add a hidden form field like the following:

```
<input name="__RequestVerificationToken" type="hidden"
value="CfDJ8NrAkSldwD9CpLRyOtm6FiJB1Jr_F3FQJQDvhlHoLNJJrLA6zaMUmhjMsisu2D2
tFkAiYgyWQawJk9vNm36sYP1esH0tamBEPvSk1_x--Sg8Ey2a-
d9CV2zHVWIN9MVhvKHOSyKqdZF1YDVd69XYx-rOWPw3i1HGLN6K0Km-
1p83jZzF0E4WU50Gg5ns2-m9Yw" />
```

## How does ASP.NET Core MVC address CSRF?

- Automatic generation of antiforgery tokens for HTML form elements can be disabled:

```
<form method="post" asp-antiforgery="false">
 ...
</form>
```

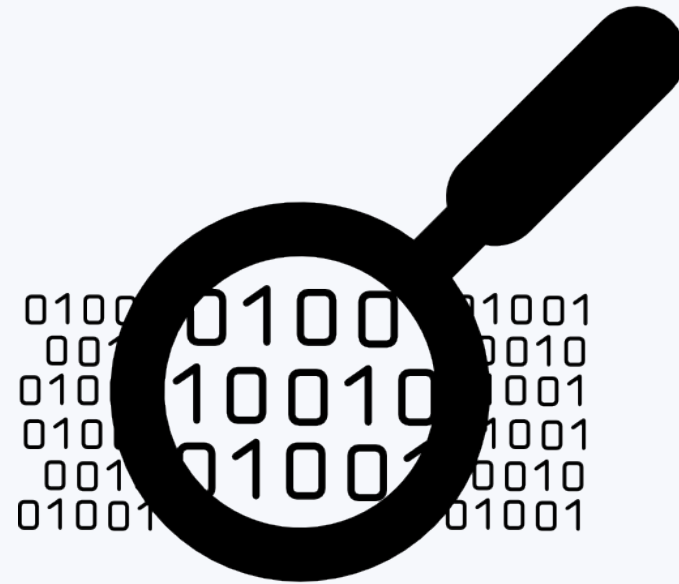
```
<!form method="post">
 ...
</!form>
```

## How does ASP.NET Core MVC address CSRF?

- [ValidateAntiForgeryToken]
  - action filter that can be applied to an individual action, a controller, or globally for the app. Requests made to actions that have this filter applied will be blocked unless the request includes a valid antiforgery token.

```
[Authorize]
public class AccountController : Controller {
 [HttpPost]
 [AllowAnonymous]
 [ValidateAntiForgeryToken]
 public async Task<IActionResult> Login(LoginModel details) {
 return View(details);
 }
}
```

# Demo



## How does ASP.NET Core MVC address CSRF?

- [AutoValidateAntiforgeryToken]
  - in most cases, your application will not receive antiforgery tokens for certain kinds of HTTP requests, such as **GET** requests. Instead of broadly applying the `ValidateAntiForgeryToken` attribute and then overriding it with `IgnoreAntiforgeryToken` attributes, you can use the `AutoValidateAntiforgeryToken` attribute.
  - This attribute works identically to the `ValidateAntiForgeryToken` attribute, except that it doesn't require tokens for requests made using the following HTTP methods: GET, HEAD, OPTIONS, TRACE
  - The previously mentioned HTTP verbs should not change the data in your application anyway

## How does ASP.NET Core MVC address CSRF?

- [IgnoreAntiforgeryToken]
  - used to eliminate the need for an antiforgery token to be present for a given action (or controller). The filter will override ValidateAntiForgeryToken and/or AutoValidateAntiforgeryToken filters specified at a higher level (globally or on a controller).

```
[Authorize]
[AutoValidateAntiforgeryToken]
public class ManageController : Controller{
 [HttpPost]
 [IgnoreAntiforgeryToken]
 public async Task<IActionResult> DoSomethingSafe(SomeViewModel model){
 // no antiforgery token required
 }
}
```

## How does ASP.NET Core MVC address CSRF?

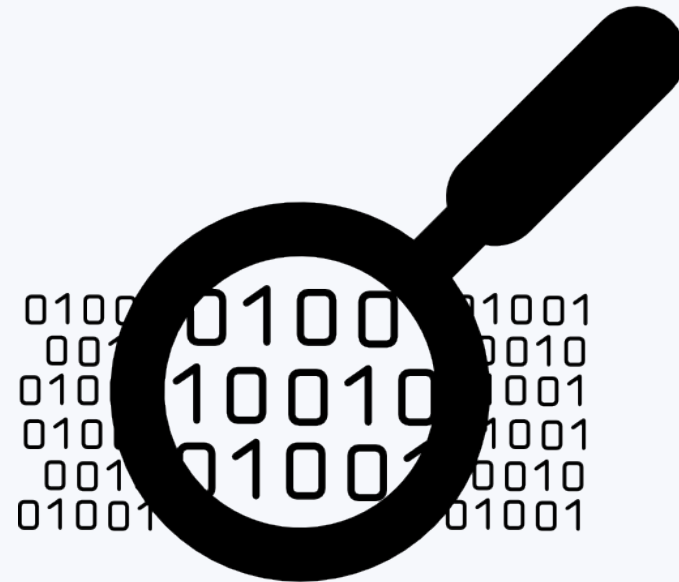
- JavaScript, AJAX, and SPAs (including Angular): [link](#)
- Further reading: <https://docs.microsoft.com/en-us/aspnet/core/security/anti-request-forgery>

# Cross-Origin Requests (CORS)

# Cross-Origin Requests (CORS)

- Browser security prevents a web page from making AJAX requests to another domain. This restriction is called the same-origin policy, and prevents a malicious site from reading sensitive data from another site. However, sometimes you might want to let other sites make cross-origin requests to your web API.
- Cross Origin Resource Sharing (CORS) is a W3C standard that allows a server to relax the same-origin policy. Using CORS, a server can explicitly allow some cross-origin requests while rejecting others. CORS is safer and more flexible than earlier techniques such as JSONP.
- Further reading: <https://docs.microsoft.com/en-us/aspnet/core/security/cors>

# Demo



# Cross-Site Scripting (XSS)

# Cross-Site Scripting (XSS)

- Cross-Site Scripting (XSS) is a security vulnerability which enables an attacker to place client-side scripts (usually JavaScript) into web pages. When other users load affected pages the attackers scripts will run, enabling the attacker to steal cookies and session tokens, change the contents of the web page through DOM manipulation or redirect the browser to another page.
- XSS vulnerabilities generally occur when an application takes user input and outputs it in a page without validating, encoding or escaping it.
- Further reading: <https://docs.microsoft.com/en-us/aspnet/core/security/cross-site-scripting>

Security – Other recommendations

# Secrets storage

Don't store secrets:

- In the application source code (it can be easily decompiled);
- In "appsettings.json";

How to store secrets:

- <https://learn.microsoft.com/en-us/aspnet/core/security/key-vault-configuration>

# General recommendations

- Only **Action** methods should be **public** in **controller** classes. The **[NonAction]** attribute can also be used to hide public methods.
- Implicit **model binding** can be used by attackers to modify unintended data.

## Other recommendations

<https://learn.microsoft.com/en-us/aspnet/core/security/>

<https://owasp.org/www-project-top-ten/>

Thank you!

Open redirect attacks



**Department of Economic Informatics and Cybernetics**  
Bucharest University of Economic Studies

# Web and Cloud Security

Microsoft Azure



# About me

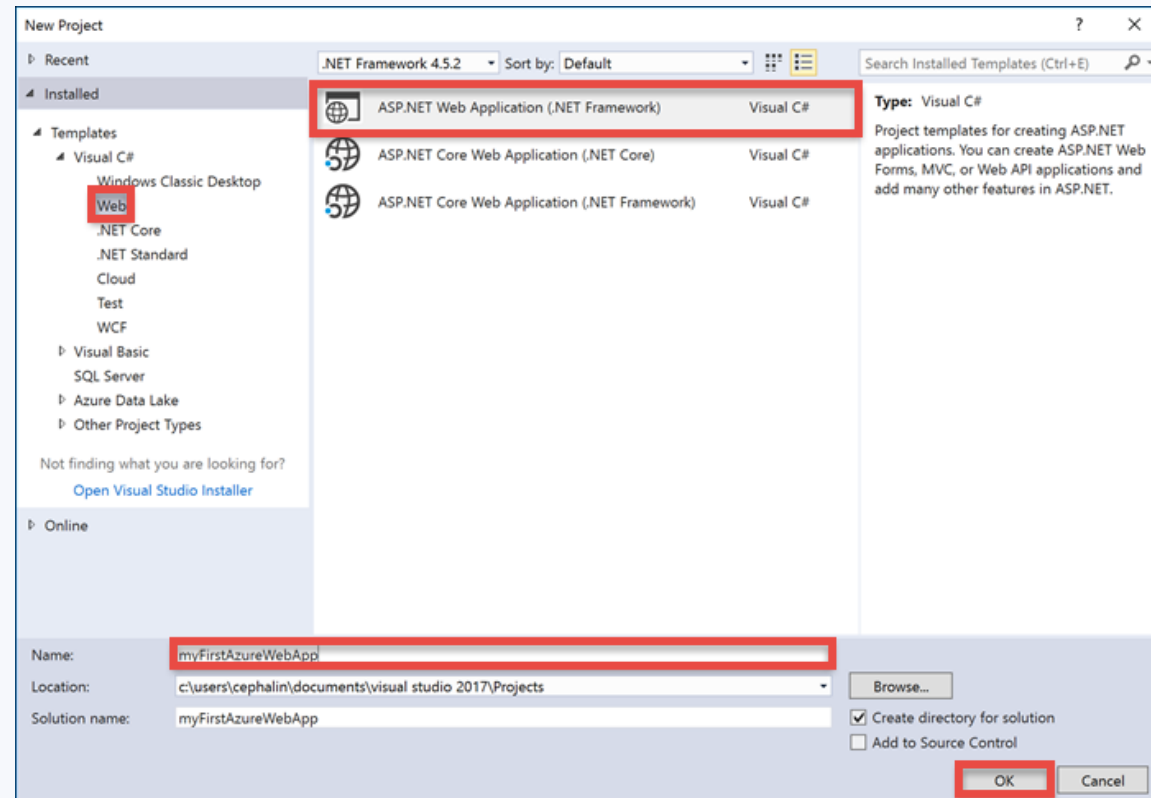


<https://ro.linkedin.com/in/cotfasliviu>

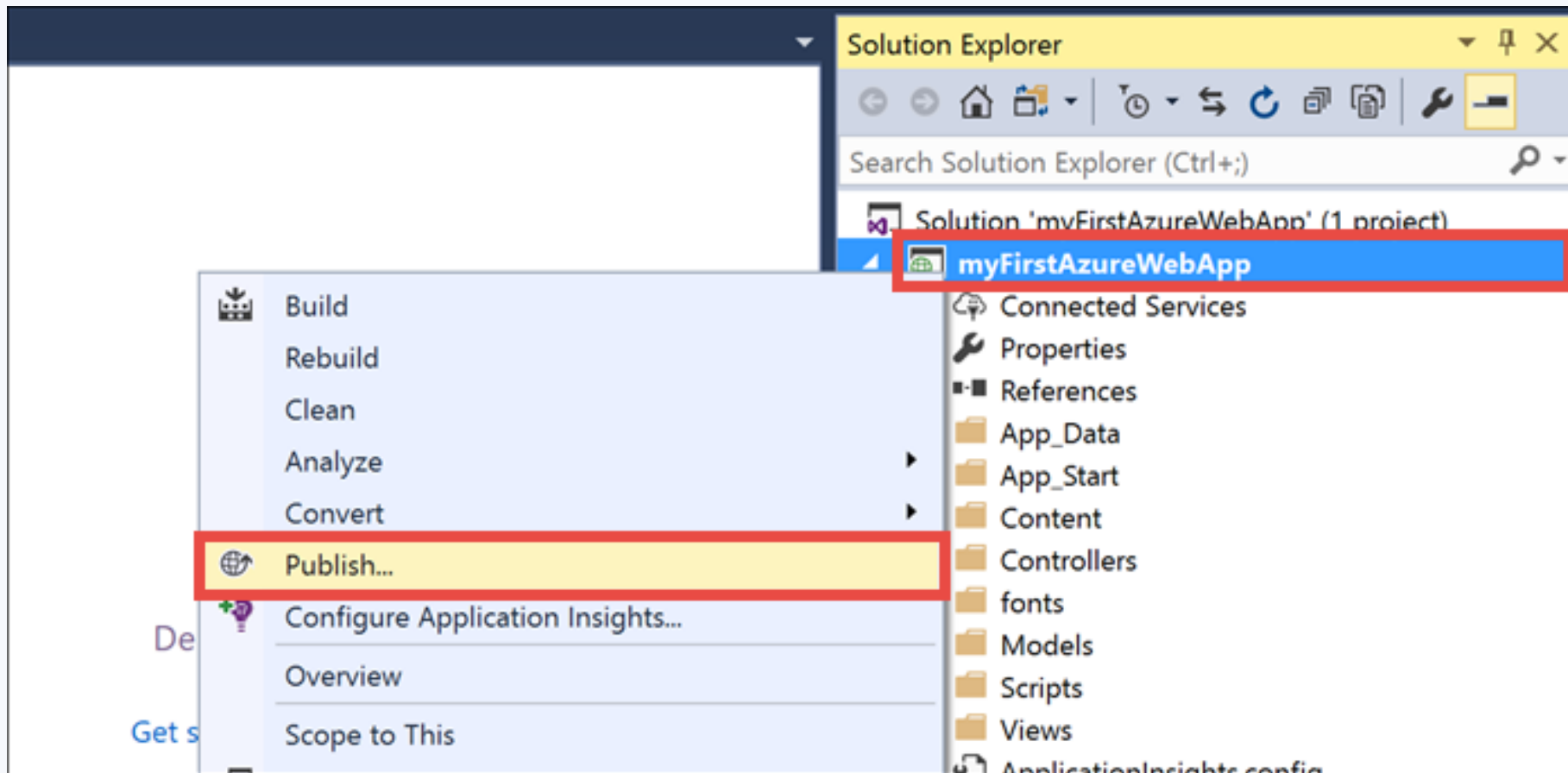
# Azure App Service

# Azure App Service

Further reading: <https://docs.microsoft.com/en-us/azure/app-service-web/app-service-web-get-started-dotnet>



# Azure App Service



# Azure App Service

## Publish

Publish your app to Azure or another host. [Learn more](#)



Microsoft Azure  
App Service



IIS, FTP, etc



Folder

>

☒ Create New

☐ Select Existing

Publish

# Azure App Service

Create App Service

Host your web and mobile applications, REST APIs, and more in Azure

Microsoft account

Hosting ⓘ

Services

Web App Name

myFirstAzureWebApp20170328111534

Change Type ▾

Subscription

Visual Studio Ultimate with MSDN ▾

Resource Group

myResourceGroup\* ▾

New... ⓘ

App Service Plan

myAppServicePlan\* ▾

New...

Clicking the Create button will create the following Azure resources

[Explore additional Azure services](#)

App Service - myFirstAzureWebApp20170328111534

App Service Plan - myAppServicePlan

If you have removed your spending limit or you are using Pay as You Go, there may be monetary impact if you provision additional resources.

[Learn More](#)

Export...

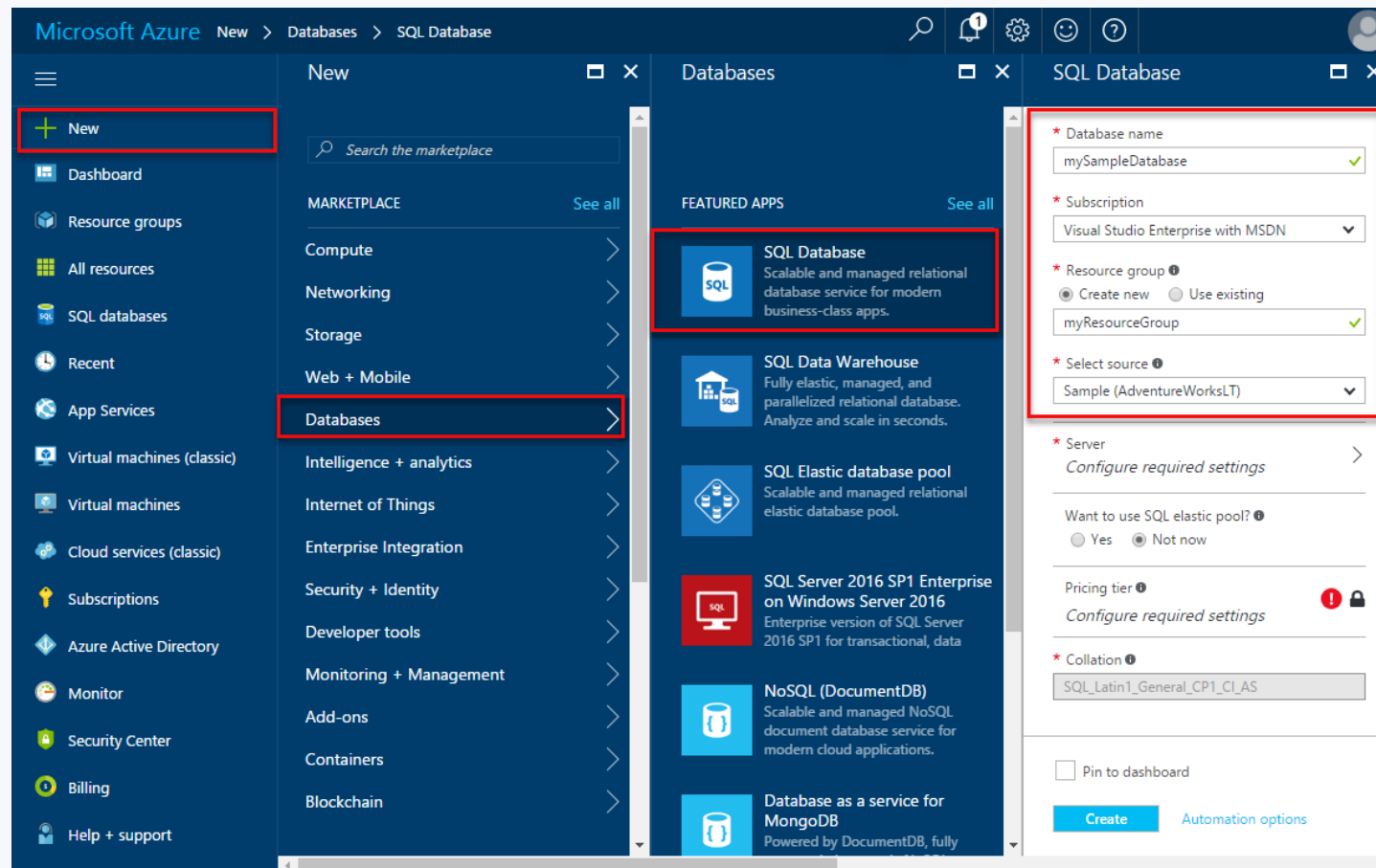
Create

Cancel

SQL Sever

# Create an Azure SQL database in the Azure portal

Further reading: <https://docs.microsoft.com/en-us/azure/sql-database/sql-database-get-started-portal>



# Create an Azure SQL database in the Azure portal

The screenshot displays the Azure portal interface for creating a new SQL database. The browser address bar shows the URL `https://portal.azure.com/#create/Microsoft.SQLDatabase`. The navigation pane on the left includes links to New, Dashboard, Resource groups, All resources, SQL databases, Recent, App Services, Virtual machines (classic), Virtual machines, Cloud services (classic), Subscriptions, Azure Active Directory, Monitor, Security Center, Billing, and Help + support.

The main content area is divided into three panels:

- SQL Database**: Contains fields for Database name (mySampleDatabase), Subscription (Visual Studio Enterprise with MSDN), Resource group (myResourceGroup), and Select source (Sample (AdventureWorksLT)). A red box highlights the "Server" section with the text "Configure required settings". Below this are options for "Want to use SQL elastic pool?" (Not now selected), "Pricing tier" (Configure required settings), and "Collation" (SQL\_Latin1\_General\_CP1\_CI\_AS). A "Create" button is at the bottom.
- Server**: Displays a "Create a new server" button (highlighted with a red box) and a message "No servers found".
- New server**: Contains fields for Server name (mynewserver-20170411), Server admin login (ServerAdmin), Password, Confirm password, and Location (West Europe). A red box highlights the "Server name" field. A "Select" button is at the bottom.

Create an Azure SQL database in the Azure portal

## Create a server-level firewall rule

The screenshot shows the Microsoft Azure portal interface for configuring a firewall rule on an SQL server named 'softscape'. The breadcrumb navigation at the top indicates the path: 'Microsoft Azure > SQL databases > ase-gisdata > softscape - Firewall'.

**Left Sidebar:**

- Navigation icons for various Azure services.
- Search bar: 'Search (Ctrl+Q)'
- Overview
- Activity log
- Access control (IAM)
- Tags
- Diagnose and solve problems
- SETTINGS**
- Quick start
- Firewall** (highlighted)
- Failover groups
- Long-term backup retention

**Main Content Area:**

Buttons: Save, Discard, Add client IP

Information box: Connections from the IPs specified below provides access to all the databases in softscape.

Allow access to Azure services: ☒ ON ☐ OFF

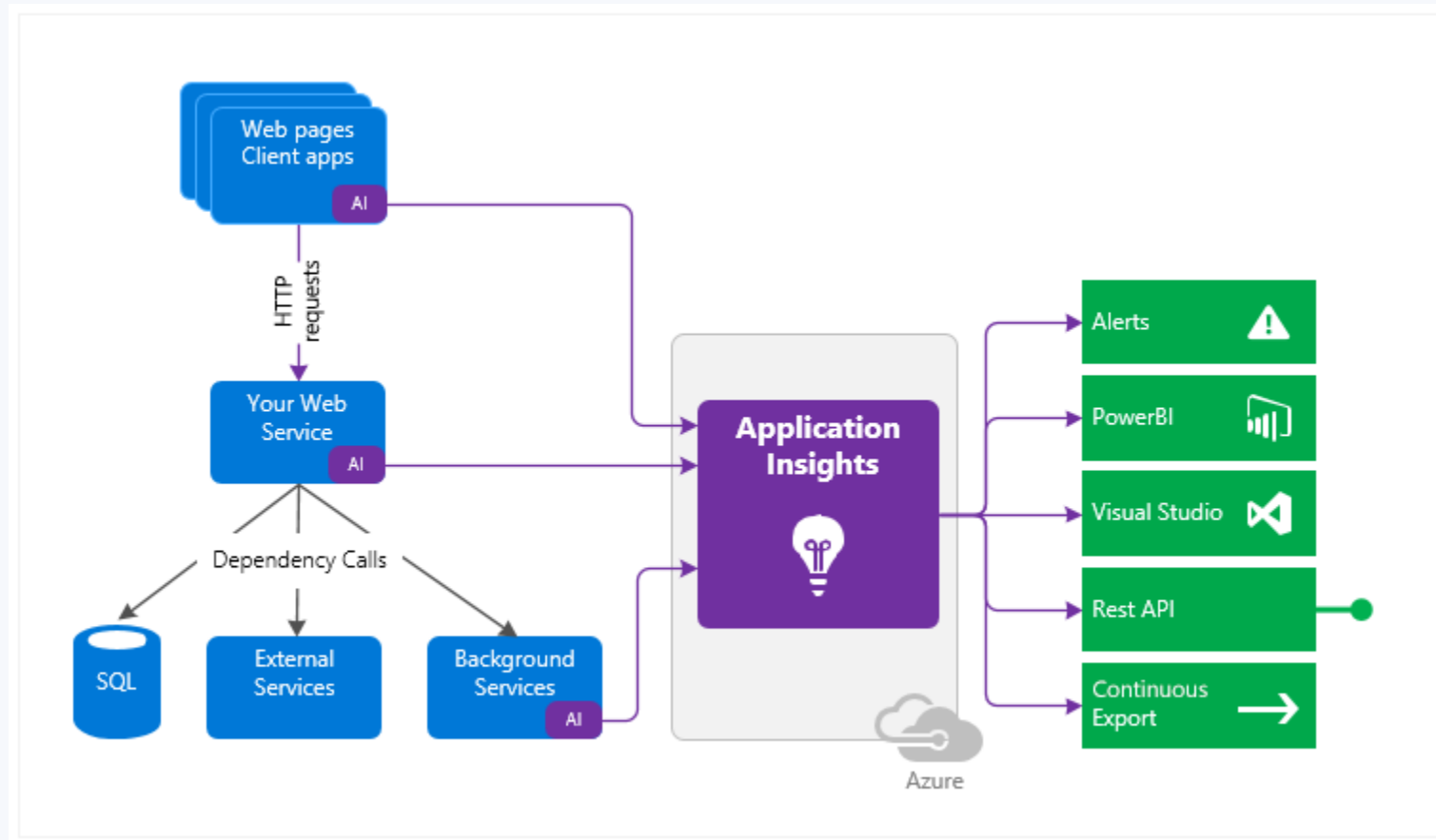
Client IP address: [Redacted]

RULE NAME	START IP	END IP
[Redacted]	[Redacted]	[Redacted]
[Redacted]	[Redacted]	[Redacted]
[Redacted]	[Redacted]	[Redacted]

# Application Insights

# Application Insights

Further reading: [What is Azure Application Insights? - Azure Monitor | Microsoft Docs](#)



Thank you!